

```
1  ===== FILE: init_project.m =====
2  function init_project()
3  % =====
4  % 文件名: init_project.m
5  % 路径: S-Function_14/init_project.m
6  % 版本号: V1.0
7  % 最后修改时间: 2025-12-10
8  % 作者: Auto-generated
9  % 功能描述:
10 % 初始化 LPV-MPC 项目运行环境:
11 % 1) 调用 project_root() 获取根目录
12 % 2) 将 src 及其子目录递归加入 MATLAB 路径
13 % 3) 将 simulink 目录加入路径, 便于加载模型
14 % =====
15
16 root = project_root();
17 addpath(root);
18 addpath(genpath(fullfile(root, 'src')));
19 addpath(fullfile(root, 'simulink'));
20
21 fprintf('[init_project] Root: %s\n', root);
22 end
23
24 ===== FILE: project_root.m =====
25 function root = project_root()
26 % =====
27 % 文件名: project_root.m
28 % 路径: S-Function_14/project_root.m
29 % 版本号: V1.0
30 % 最后修改时间: 2025-12-10
31 % 作者: Auto-generated
32 % 功能描述:
33 % 返回项目根目录的绝对路径, 供所有脚本构建相对路径。
34 % 该函数应位于项目根目录, 不随其他脚本迁移。
35 % =====
36
37 persistent cached_root
38 if isempty(cached_root) || ~isfolder(cached_root)
39     [cached_root, ~] = fileparts(mfilename('fullpath'));
40 end
41 root = cached_root;
42 end
43
44 ===== FILE: results_dir.m =====
45 function dir_out = results_dir(subdir)
46 % =====
47 % 文件名: results_dir.m
48 % 路径: S-Function_14/results_dir.m
49 % 版本号: V1.0
50 % 最后修改时间: 2025-12-10
```

```

1  % 作者: Auto-generated
2  % 功能描述:
3  %   构建结果输出目录的绝对路径, 并在不存在时自动创建。
4  %   示例: out_dir = results_dir('gru/performance_offline');
5  % =====
6
7  if nargin < 1 || isempty(subdir)
8      subdir = '';
9  end
10
11 root = project_root();
12 dir_out = fullfile(root, 'results', subdir);
13
14 if ~exist(dir_out, 'dir')
15     mkdir(dir_out);
16 end
17 end
18
19 ===== FILE: src/core/agv_model_sfunc.m =====
20
21 function agv_model_sfunc(block)
22 % =====
23 % Level-2 MATLAB S-Function (Simulink)
24 % 外壳: AGV 离散植物, 内部调用 state_eq / output_eq
25 % 接口:
26 %   - 参数 p: parameters() 结构体 (必须包含 Ts; 可选 nx、x0)
27 %   - 输入端口(1): u_all = [F_cmd; omega_cmd; theta_ground] (3x1)
28 %   - 输出端口(1): y (34x1) - V4.3 Mamba 适配
29 %   - 离散状态: x (nx×1, 默认 8), 使用 DWork 管理
30 % =====
31 setup(block);
32
33 % ----- S-Function 框架 -----
34 function setup(block)
35     % 1) 模块参数
36     block.NumDialogPrms = 1;    % p = parameters()
37
38     % 2) 端口
39     block.NumInputPorts = 1;
40     block.NumOutputPorts = 1;
41
42     % 输入端口: 3×1
43     block.SetPreCompInpPortInfoToDynamic;
44     block.InputPort(1).Dimensions = 3;
45     block.InputPort(1).DatatypeID = 0; % double
46     block.InputPort(1).Complexity = 'Real';
47     block.InputPort(1).DirectFeedthrough = false;
48
49     % 输出端口: 34×1 (V4.3 Mamba 适配)
50     block.SetPreCompOutPortInfoToDynamic;

```

```

1      block.OutputPort(1).Dimensions = 34;
2      block.OutputPort(1).DatatypeID = 0; % double
3      block.OutputPort(1).Complexity = 'Real';
4
5      % 3) 采样时间 (来自参数 Ts)
6      p = block.DialogPrm(1).Data;
7      if ~isfield(p,'Ts')
8          error('parameters() 必须包含字段 Ts');
9      end
10     block.SampleTimes = [p.Ts 0];
11
12     % 4) 不在 setup 里设置 NumDworks / Dwork 属性
13
14     % 5) 注册方法
15     block.SimStateCompliance = 'DefaultSimState';
16     block.RegBlockMethod('PostPropagationSetup', @DoPostPropSetup);
17     block.RegBlockMethod('InitializeConditions', @DoInitialize);
18     block.RegBlockMethod('Outputs', @DoOutputs);
19     block.RegBlockMethod('Update', @DoUpdate);
20 end
21
22 % ----- 在 PostPropagationSetup 里设置 DWork 的数量和属性 -----
23 function DoPostPropSetup(block)
24     block.NumDworks = 1; % 关键: 必须在这里设置
25
26     % 状态维度 (可用 p.nx 指定, 默认 8)
27     nx = 8;
28     if block.NumDialogPrms>=1 && isstruct(block.DialogPrm(1).Data)
29         p = block.DialogPrm(1).Data;
30         if isfield(p,'nx') && ~isempty(p.nx)
31             nx = p.nx;
32         end
33     end
34
35     block.Dwork(1).Name = 'xdisc';
36     block.Dwork(1).Dimensions = nx;
37     block.Dwork(1).DatatypeID = 0; % double
38     block.Dwork(1).Complexity = 'Real';
39     block.Dwork(1).UsedAsDiscState = true; % 离散状态
40 end
41
42 % ----- 初始化离散状态 -----
43 function DoInitialize(block)
44     p = block.DialogPrm(1).Data;
45     nx = block.Dwork(1).Dimensions;
46
47     % 优先使用 p.x0; 否则由 parameters.m 的 initial_* 字段组装
48     if isfield(p,'x0') && ~isempty(p.x0)
49         x0 = p.x0(:);
50     elseif all(isfield(p, {'initial_x','initial_y','initial_heading', ...

```

```

1         'initial_velocity','initial_angular_velocity', ...
2         'initial_front_steering','initial_rear_steering', ...
3         'initial_sideslip'}))
4     x0 = [p.initial_x; p.initial_y; p.initial_heading; ...
5           p.initial_velocity; p.initial_angular_velocity; ...
6           p.initial_front_steering; p.initial_rear_steering; ...
7           p.initial_sideslip];
8     else
9         x0 = zeros(nx,1);
10    end
11
12    if numel(x0)~=nx
13        error('x0 维度(%d)与 nx(%d)不一致', numel(x0), nx);
14    end
15    block.Dwork(1).Data = x0;
16 end
17
18 % ----- 输出方程 -----
19 function DoOutputs(block)
20     p = block.DialogPrm(1).Data;
21     u_all = block.InputPort(1).Data; % [F_cmd; omega_cmd; theta_ground]
22     x = block.Dwork(1).Data;
23
24     u = u_all(1:2); % 控制量
25     theta_ground = u_all(3); % 坡度角
26
27     % y = output_eq(x, u, theta_ground, p); % 修正: 按签名传入 4 个参数
28     y = output_eq_ref(x, u, theta_ground, p); % 利用 v_ref 和 omega_ref 生成参考半径
29
30     if ~isvector(y) || numel(y)~=block.OutputPort(1).Dimensions
31         % error('output_eq 返回长度与输出端口不一致, 期望 %d, 实际 %d', ... % 利用 v_ref 和
omega_ref 生成参考半径
32             % block.OutputPort(1).Dimensions, numel(y));
33         error('output_eq_ref 返回长度与输出端口不一致, 期望 %d, 实际 %d', ...
34             block.OutputPort(1).Dimensions, numel(y));
35     end
36     block.OutputPort(1).Data = y(:);
37 end
38
39 % ----- 状态更新 (离散化步进) -----
40 function DoUpdate(block)
41     p = block.DialogPrm(1).Data;
42     u_all = block.InputPort(1).Data; % [F_cmd; omega_cmd; theta_ground]
43     x = block.Dwork(1).Data;
44
45     u = u_all(1:2);
46     theta_ground = u_all(3);
47
48     % 修正: state_eq 的签名为 (x, u, theta_ground, params)
49     % x_next = state_eq(x, u, theta_ground, p); % 利用 v_ref 和 omega_ref 生成参考半径
50     x_next = state_eq_ref(x, u, theta_ground, p);

```

```

1
2     if numel(x_next)~=numel(x)
3         % error('state_eq 返回维度与 DWork 不一致');           % 利用 v_ref 和 omega_ref 生成参考半径
4         error('state_eq_ref 返回维度与 DWork 不一致');
5     end
6     block.Dwork(1).Data = x_next(:);
7 end
8 end
9
10
11 ===== FILE: src/core/agv_model_sfunc_train_data.m =====
12 function agv_model_sfunc_train_data(block)
13 %AGV_MODEL_SFUNC_TRAIN_DATA 训练数据生成专用 Level-2 MATLAB S-Function。
14 %
15 % 功能说明:
16 % 本模块由 GRU_DataGen.slx 调用, 用于生成 GRU/TCN 共享训练数据。
17 % 它与闭环主模型 agv_model_sfunc.m 分离, 原因是训练数据生成需要
18 % 额外注入滑移和负载/堵转扰动, 不能污染正式控制模型。
19 %
20 % 输入端口:
21 % u_all = [F_cmd; omega_cmd; theta_ground; slip_gamma; stall_load]
22 %
23 % F_cmd          : 纵向驱动力命令 [N]
24 % omega_cmd      : 横摆角速度命令 [rad/s]
25 % theta_ground   : 地面坡度角 [rad]
26 % slip_gamma     : 滑移注入系数, 1.0 表示正常附着
27 % stall_load     : 外加纵向负载, 用于 load/stall 扰动注入 [N]
28 %
29 % 内部动力学/输出方程:
30 % state_eq_ref_train_data(...)
31 % output_eq_ref_train_data(...)
32 %
33 % 输出约定:
34 % y 为 34 x 1, 保持当前 GRU/Mamba/TCN 数据生成链路使用的训练信号
35 % 契约。第 32-34 维为滑移率和横向加速度等扩展诊断量。
36
37 setup(block);
38
39 function setup(block)
40     block.NumDialogPrms = 1; % params 参数结构体
41
42     block.NumInputPorts = 1;
43     block.NumOutputPorts = 1;
44
45     block.SetPreCompInpPortInfoToDynamic;
46     block.InputPort(1).Dimensions = 5;
47     block.InputPort(1).DatatypeID = 0; % double 类型
48     block.InputPort(1).Complexity = 'Real';
49     block.InputPort(1).DirectFeedthrough = false;
50

```

```

1      block.SetPreCompOutPortInfoToDynamic;
2      block.OutputPort(1).Dimensions = 34;
3      block.OutputPort(1).DatatypeID = 0; % double 类型
4      block.OutputPort(1).Complexity = 'Real';
5
6      p = block.DialogPrm(1).Data;
7      if ~isstruct(p) || ~isfield(p, 'Ts')
8          error('agv_model_sfnc_train_data:InvalidParams', ...
9              'Dialog parameter must be a params struct with field Ts.');
```

10 end

11 block.SampleTimes = [p.Ts 0];

12

13 block.SimStateCompliance = 'DefaultSimState';

14 block.RegBlockMethod('PostPropagationSetup', @DoPostPropSetup);

15 block.RegBlockMethod('InitializeConditions', @DoInitialize);

16 block.RegBlockMethod('Outputs', @DoOutputs);

17 block.RegBlockMethod('Update', @DoUpdate);

18 end

19

20 function DoPostPropSetup(block)

21 nx = 8;

22 if block.NumDialogPrms >= 1 && isstruct(block.DialogPrm(1).Data)

23 p = block.DialogPrm(1).Data;

24 if isfield(p, 'nx') && ~isempty(p.nx)

25 nx = p.nx;

26 end

27 end

28

29 block.NumDworks = 1;

30 block.Dwork(1).Name = 'xdisc';

31 block.Dwork(1).Dimensions = nx;

32 block.Dwork(1).DatatypeID = 0;

33 block.Dwork(1).Complexity = 'Real';

34 block.Dwork(1).UsedAsDiscState = true;

35 end

36

37 function DoInitialize(block)

38 p = block.DialogPrm(1).Data;

39 nx = block.Dwork(1).Dimensions;

40

41 if isfield(p, 'x0') && ~isempty(p.x0)

42 x0 = p.x0(:);

43 elseif all(isfield(p, {'initial_x', 'initial_y', 'initial_heading', ...

44 'initial_velocity', 'initial_angular_velocity', ...

45 'initial_front_steering', 'initial_rear_steering', ...

46 'initial_sideslip'}))

47 x0 = [p.initial_x; p.initial_y; p.initial_heading; ...

48 p.initial_velocity; p.initial_angular_velocity; ...

49 p.initial_front_steering; p.initial_rear_steering; ...

50 p.initial_sideslip];

```
1      else
2          x0 = zeros(nx, 1);
3      end
4
5      if numel(x0) ~= nx
6          error('agv_model_sfunc_train_data:InvalidX0', ...
7              'x0 length (%d) does not match nx (%d).', numel(x0), nx);
8      end
9      block.Dwork(1).Data = x0;
10 end
11
12 function DoOutputs(block)
13     p = block.DialogPrm(1).Data;
14     u_all = block.InputPort(1).Data;
15     x = block.Dwork(1).Data;
16
17     u = u_all(1:2);
18     theta_ground = u_all(3);
19     slip_gamma = u_all(4);
20     stall_load = u_all(5);
21
22     y = output_eq_ref_train_data(x, u, theta_ground, p, slip_gamma, stall_load);
23
24     if ~isvector(y) || numel(y) ~= block.OutputPort(1).Dimensions
25         error('agv_model_sfunc_train_data:InvalidOutputSize', ...
26             'output_eq_ref_train_data returned %d elements, expected %d.', ...
27             numel(y), block.OutputPort(1).Dimensions);
28     end
29     block.OutputPort(1).Data = y(:);
30 end
31
32 function DoUpdate(block)
33     p = block.DialogPrm(1).Data;
34     u_all = block.InputPort(1).Data;
35     x = block.Dwork(1).Data;
36
37     u = u_all(1:2);
38     theta_ground = u_all(3);
39     slip_gamma = u_all(4);
40     stall_load = u_all(5);
41
42     x_next = state_eq_ref_train_data(x, u, theta_ground, p, slip_gamma, stall_load);
43
44     if numel(x_next) ~= numel(x)
45         error('agv_model_sfunc_train_data:InvalidStateSize', ...
46             'state_eq_ref_train_data returned %d elements, expected %d.', ...
47             numel(x_next), numel(x));
48     end
49     block.Dwork(1).Data = x_next(:);
50 end
```

```

1
2   end
3
4   ===== FILE: src/core/state_eq.m =====
5   function x_next = state_eq(x, u, theta_ground, params)
6   % =====
7   % 文件名: state_eq.m
8   % 功能描述: 对角式双舵轮 AGV 状态转移方程 - 实现车辆动力学模型的时间步进
9   % 输入参数:
10  %   x: 当前时刻状态向量 [X Y psi v omega delta_lf delta_rr beta] (8×1)
11  %       其中: X-全局 X 坐标[m], Y-全局 Y 坐标[m], psi-航向角[rad]
12  %           v-纵向速度[m/s], omega-横摆角速度[rad/s]
13  %           delta_lf-左前轮转向角[rad], delta_rr-右后轮转向角[rad]
14  %           beta-质心侧偏角[rad]
15  %   u: 控制输入向量 [F_cmd omega_cmd] (2×1)
16  %       F_cmd-总驱动力指令[N], omega_cmd-期望横摆角速度[rad/s]
17  %   theta_ground: 地面坡度角[rad] (标量)
18  %   params: 包含所有车辆参数的结构体
19  % 输出参数:
20  %   x_next: 下一时刻状态向量 (8×1)
21  % =====
22
23  %% 提取状态变量 - 从状态向量中分解各个状态分量
24  X = x(1); Y = x(2); psi = x(3); v = x(4);
25  omega = x(5); delta_lf = x(6); delta_rr = x(7); beta = x(8);
26
27  %% 提取控制输入 - 从输入向量分解控制指令
28  F_cmd = u(1); % 总驱动力指令[N]
29  omega_cmd = u(2); % 期望横摆角速度指令[rad/s]
30
31  %% 直接从参数结构体获取车辆参数 - 提高代码可读性和维护性
32  Ts = params.Ts; % 采样时间[s]
33  m = params.mass; % 车辆总质量[kg]
34  I_z = params.Iz; % 绕 Z 轴的转动惯量[kg·m²]
35  L = params.L; % 轴距[m]
36  W = params.W; % 轮距[m]
37  h_cg = params.h_cg; % 质心高度[m]
38  r = params.wheel_radius; % 车轮半径[m]
39  n = params.gear_ratio; % 传动比
40  eta = params.gear_efficiency; % 传动效率
41  k_t = params.motor_torque_constant; % 电机转矩常数[N·m/A]
42  mu = params.friction_coefficient; % 路面摩擦系数
43  c_r = params.rolling_resistance; % 滚动阻力系数
44  rho = params.air_density; % 空气密度[kg/m³]
45  CdA = params.drag_coefficient_area; % 风阻系数×迎风面积[m²]
46  C_af = params.front_cornering_stiffness; % 前轮侧偏刚度[N/rad]
47  C_ar = params.rear_cornering_stiffness; % 后轮侧偏刚度[N/rad]
48  delta_max = params.max_steering_angle; % 最大转向角限制[rad]
49  delta_dot_max = params.max_steering_rate; % 最大转向速率[rad/s]
50  tau_delta = params.steering_time_constant; % 转向系统时间常数[s]

```



```

1  current_limit = params.current_limit;    % 电机电流限制[A]
2  accel_limit = params.max_acceleration;   % 最大加速度限制[m/s²]
3  g = params.gravity;                     % 重力加速度[m/s²]
4  min_omega_threshold = params.min_angular_velocity_threshold; % 最小角速度阈值[rad/s]
5  low_speed_thresh = params.low_speed_threshold; % 低速阈值[m/s]
6
7  %% === 转向几何计算: 按 ICR 约束 (tanδ 关系) 生成目标角 ===
8  if abs(omega_cmd) < min_omega_threshold || v < low_speed_thresh
9      % 低速/小指令: 维持直行
10     delta_lf_target = 0; delta_rr_target = 0;
11 else
12     % 角速度误差抑制 (比例缩放, 保持两轮几何比不变)
13     e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]
14     if abs(omega) > abs(omega_cmd) * 1.2 && sign(omega) == sign(omega_cmd)
15         delta_scale = max(omega_cmd / max(abs(omega), 1e-6), 0.1);
16     else
17         delta_scale = 1.0;
18     end
19
20     % ICR 坐标 (车辆质心坐标系): 改用测量 ω (或与命令加权的 ω) 计算
21     % 目的: 避免在 v 下降或跟踪误差较大时, 把目标舵角放大
22     w_icr = omega; % 如需更强前馈, 可用 w_icr = 0.7*omega + 0.3*omega_cmd;
23     R_cmd = (v / max(abs(w_icr), 1e-6)) * sign(w_icr); % 带符号半径[m]
24     x_c = 0.0; % 对称转弯, ICR 在 y 轴上
25     y_c = R_cmd; % ICR y 坐标 (可正可负)
26
27     % 几何约束:
28     % tan(delta_lf) = (L/2 - x_c) / (y_c - W/2)
29     % tan(delta_rr) = -(x_c + L/2) / (y_c + W/2)
30     delta_lf_geom = atan2((L/2 - x_c), (y_c - W/2));
31     delta_rr_geom = -atan2((x_c + L/2), (y_c + W/2));
32
33     % 误差缩放 (不破坏两轮几何比例)
34     delta_lf_target = delta_lf_geom * delta_scale;
35     delta_rr_target = delta_rr_geom * delta_scale;
36
37     % 取消额外的“安全夹紧”: 由执行器限幅 delta_max 与速率限幅保障
38     % 这样可避免稳态曲率下目标转角长期贴住 15° 而过大
39 end
40
41 %% === 舵机动力学模拟: 一阶惯性 + 速率限制 + 角度限制 ===
42 % 转向系统动力学: 一阶惯性环节模拟转向执行器响应
43 delta_lf_dot = (delta_lf_target - delta_lf)/max(tau_delta, Ts);
44 delta_rr_dot = (delta_rr_target - delta_rr)/max(tau_delta, Ts);
45
46 % 转向速率限制 - 模拟转向执行器的物理限制
47 delta_lf_dot = sat(delta_lf_dot, -delta_dot_max, delta_dot_max);
48 delta_rr_dot = sat(delta_rr_dot, -delta_dot_max, delta_dot_max);
49
50 % 转向角更新 + 角度限制

```

```

1  delta_lf_new = sat(delta_lf + Ts*delta_lf_dot, -delta_max, delta_max);
2  delta_rr_new = sat(delta_rr + Ts*delta_rr_dot, -delta_max, delta_max);
3
4  %% === 载荷转移计算 & 滚动阻力计算 ===
5  [N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_ground,
params);
6
7  %% === 电机/传动系统限制推算到车轮牵引力上限 ===
8  % 计算单个驱动轮的最大牵引力（基于电机电流限制）
9  F_wheel_max = current_limit * k_t * (eta * n) / max(r,1e-6); % 单驱动轮最大牵引力[N]
10
11 % 计算总驱动轮的最大牵引力（对角式：左前+右后两个驱动轮）
12 num_drive_wheels = 2; % 驱动轮数量: lf & rr
13 F_cmd_max_total = num_drive_wheels * F_wheel_max;
14
15 % 应用牵引力限制到指令值
16 F_cmd_eff = sat_sym(F_cmd, F_cmd_max_total);
17
18 %% === 轮胎侧偏角计算（基于简化自行车模型） ===
19 % 车辆几何参数：假设质心位于轴距中心
20 Lf = L/2; Lr = L/2; % 前后轴到质心的距离[m]
21
22 % 侧偏刚度：使用标称值（取消转弯增强的临时放大）
23 % C_af, C_ar 已由 parameters.m 提供
24
25 % 纵向速度分量（防止除零）
26 v_x = max(v * cos(beta), 1e-3);
27
28 % 前轮侧偏角（标准符号）: alpha_f = (beta + Lf*omega/v) - delta_lf
29 alpha_f = (beta + Lf*omega/max(v,low_speed_thresh)) - delta_lf_new;
30
31 % 后轮侧偏角（标准符号）: alpha_r = (beta - Lr*omega/v) - delta_rr
32 alpha_r = (beta - Lr*omega/max(v,low_speed_thresh)) - delta_rr_new; % 对角式双舵轮，后轮也有转向
33
34 % 线性侧偏力计算 + 软饱和（限幅到摩擦圆边界）
35 Fy_f_lin = -C_af * alpha_f; % 前轮线性侧偏力[N]
36 Fy_r_lin = -C_ar * alpha_r; % 后轮线性侧偏力[N]
37
38 % 基于垂向载荷计算最大侧向力（摩擦圆限制）
39 Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
40 Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]
41
42 % 应用侧向力饱和限制
43 Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
44 Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
45
46 %% === 牵引力分配策略：按法向载荷比例 + 偏航控制 ===
47 % 驱动轮总垂向载荷（仅考虑两个驱动轮：左前+右后）
48 W_drive = N_lf + N_rr; % 驱动轮总载荷[N]
49
50 % 计算各驱动轮在总驱动载荷中的权重比例

```

```

1  w_lf = N_lf / max(W_drive,1e-6); % 左前轮载荷占比
2  w_rr = N_rr / max(W_drive,1e-6); % 右后轮载荷占比
3
4  % 直线/低曲率时等分扭矩，抑制对角驱动导致的偏航力矩
5  if (abs(omega_cmd) < min_omega_threshold) && (abs(omega) < min_omega_threshold)
6      w_lf = 0.5;
7      w_rr = 0.5;
8  end
9
10 % 偏航控制：根据角速度误差产生驱动力差以实现横摆控制
11 e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]
12
13 % 横摆角速度保护：防止角速度爆炸式增长
14 omega_max = 2.0; % 最大角速度限制 [rad/s] (远大于目标 0.1 rad/s)
15
16 if v < low_speed_thresh || abs(omega_cmd) < min_omega_threshold
17     % 低速/小指令：关断偏航力差，避免原地自旋
18     Delta_Fx = 0;
19 else
20     if abs(omega) > omega_max
21         % 情况：角速度超限 - 施加强制制动力矩进行保护
22         omega_sign = sign(omega);
23         Mz_brake = -omega_sign * I_z * 50.0; % 强制制动力矩[Nm]
24         Delta_Fx = 2 * Mz_brake / max(W, 1e-3); % 所需的驱动力差[N]
25     else
26         % 恢复：正常 P 控制产生偏航力差
27         K_omega_p = 50.0;
28         omega_dot_desired = K_omega_p * e_omega; % 期望角加速度[rad/s²]
29         omega_dot_max = 10.0; % 最大角加速度 [rad/s²]
30         omega_dot_desired = sat_sym(omega_dot_desired, omega_dot_max);
31         Mz_needed = I_z * omega_dot_desired; % 所需偏航力矩 [Nm]
32         Delta_Fx = 2 * Mz_needed / max(W, 1e-3); % 所需驱动力差 [N]
33         Delta_Fx_max = max(abs(F_cmd_eff), 10) * 0.5; % 驱动力差限幅
34         Delta_Fx = sat_sym(Delta_Fx, Delta_Fx_max);
35     end
36 end
37
38 % 驱动力分配：基础分配（按载荷比例）+ 偏航控制（力差调整）
39 Fx_lf_cmd = F_cmd_eff * w_lf + Delta_Fx/2; % 左前轮指令驱动力[N]
40 Fx_rr_cmd = F_cmd_eff * w_rr - Delta_Fx/2; % 右后轮指令驱动力[N]
41
42 %% === 摩擦椭圆限幅：确保合力不超过轮胎-路面摩擦极限 ===
43 % 获取是否使用光滑限幅的标志
44 use_smooth = params.use_smooth_saturation;
45
46 if use_smooth
47     % 光滑版本限幅：使用 softplus 函数替代 max(·,0)，提高数值稳定性
48     smooth_gain = params.smooth_gain;
49
50     % 计算各轮胎在摩擦椭圆中的可用纵向力余量

```

```

1      term_lf = softplus(1 - (Fy_f/max(mu*N_lf,1e-6))^2, smooth_gain);
2      term_rr = softplus(1 - (Fy_r/max(mu*N_rr,1e-6))^2, smooth_gain);
3
4      Fx_lf_allow = mu * N_lf * sqrt(term_lf); % 左前轮允许的最大纵向力[N]
5      Fx_rr_allow = mu * N_rr * sqrt(term_rr); % 右后轮允许的最大纵向力[N]
6
7      % 应用光滑限幅到指令驱动力
8      F_drive_lf = sat_smooth(Fx_lf_cmd, -Fx_lf_allow, Fx_lf_allow, smooth_gain);
9      F_drive_rr = sat_smooth(Fx_rr_cmd, -Fx_rr_allow, Fx_rr_allow, smooth_gain);
10     else
11         % 硬限幅版本: 传统的摩擦椭圆限制
12         Fx_lf_allow = mu * N_lf * sqrt(max(1 - (Fy_f/max(mu*N_lf,1e-6))^2, 0));
13         Fx_rr_allow = mu * N_rr * sqrt(max(1 - (Fy_r/max(mu*N_rr,1e-6))^2, 0));
14
15         F_drive_lf = sign(Fx_lf_cmd) * min(abs(Fx_lf_cmd), Fx_lf_allow);
16         F_drive_rr = sign(Fx_rr_cmd) * min(abs(Fx_rr_cmd), Fx_rr_allow);
17     end
18
19     % 计算总有效驱动力
20     F_drive_total = F_drive_lf + F_drive_rr;
21
22     % 计算驱动不对称产生的偏航力矩（左右驱动力差异引起）
23     Mz_drive = (W/2) * (F_drive_lf - F_drive_rr);
24
25     % 坡度阻力计算
26     F_slope = m * g * sin(theta_ground);
27
28     % 有效质量计算（包含旋转部件的等效质量）
29     wheel_inertia = params.wheel_inertia; % 车轮转动惯量[kg·m²]
30     motor_inertia = params.motor_inertia; % 电机转动惯量[kg·m²]
31     m_eff_total = m + 2*(wheel_inertia + motor_inertia*n^2)/(r^2); % 总有效质量[kg]
32
33     %% === RK4 数值积分（转向角线性插值 + 动态空气阻力计算） ===
34     % 提取核心状态变量用于 RK4 积分
35     s_k = [X; Y; psi; v; omega; beta];
36
37     % 转向角中点值（用于 RK4 中间步骤的线性插值）
38     delta_lf_mid = (delta_lf + delta_lf_new)/2;
39     delta_rr_mid = (delta_rr + delta_rr_new)/2;
40
41     % 获取光滑限幅参数
42     use_smooth = params.use_smooth_saturation;
43     smooth_gain = params.smooth_gain;
44
45     % 定义连续动力学核心函数（闭包形式，传入当前计算的所有参数）
46     CONT_CORE = @(s, dl, dr) continuous_dynamics_core_v2(s, dl, dr, ...
47         F_drive_lf, F_drive_rr, F_rolling_total, F_slope, m_eff_total, I_z, Lf, Lr, ...
48         low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth, smooth_gain, ...
49         rho, CdA); % 传入空气阻力参数用于动态计算
50

```

```

1    % RK4 积分四步计算（使用转向角插值）
2    k1 = CONT_CORE(s_k, delta_lf, delta_rr); % 第一步：使用 k 时刻转向角
3    k2 = CONT_CORE(s_k + 0.5*Ts*k1, delta_lf_mid, delta_rr_mid); % 第二步：使用中点转向角
4    k3 = CONT_CORE(s_k + 0.5*Ts*k2, delta_lf_mid, delta_rr_mid); % 第三步：使用中点转向角
5    k4 = CONT_CORE(s_k + Ts*k3, delta_lf_new, delta_rr_new); % 第四步：使用 k+1 时刻转向角
6
7    % RK4 最终积分：加权平均
8    s_new = s_k + (Ts/6)*(k1 + 2*k2 + 2*k3 + k4);
9
10   %% 提取新状态并应用保护措施
11   X_new = s_new(1); Y_new = s_new(2); psi_new = s_new(3);
12   v_new = s_new(4); omega_new = s_new(5); beta_new = s_new(6);
13
14   % 航向角归一化到[-pi, pi]范围
15   psi_new = normalizeAngle(psi_new);
16
17   % 低速保护：防止速度变为负值（车辆不能倒车）
18   if v_new < 0
19       v_new = 0;
20   end
21
22   % 侧偏角限幅保护（防止数值爆炸）
23   beta_new = sat(beta_new, -deg2rad(15), deg2rad(15));
24
25   %% 组装下一时刻完整状态向量
26   x_next = [X_new; Y_new; psi_new; v_new; omega_new; delta_lf_new; delta_rr_new; beta_new];
27
28   end
29
30   %% ===== 连续动力学核心函数 V2（动态空气阻力计算）
31   =====
32   function ds = continuous_dynamics_core_v2(s, delta_lf, delta_rr, Fx_lf, Fx_rr, F_rolling_total,
33       F_slope, ...
34       m_eff_total, I_z, Lf, Lr, low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth,
35       smooth_gain, ...
36       rho, CdA)
37   % 连续动力学核心函数 V2 - 用于 RK4 积分的每个子步，动态计算空气阻力
38   % 输入参数：
39   % s: 状态向量 [X Y psi v omega beta]
40   % delta_lf, delta_rr: 当前转向角[rad]
41   % Fx_lf, Fx_rr: 预计算的驱动力（已应用摩擦椭圆限幅）[N]
42   % F_rolling_total: 总滚动阻力[N]
43   % F_slope: 坡度阻力[N]
44   % m_eff_total: 总有效质量[kg]
45   % I_z: 转动惯量[kg·m²]
46   % Lf, Lr: 前后轴到质心距离[m]
47   % low_speed_thresh: 低速阈值[m/s]
48   % C_af, C_ar: 前后轮侧偏刚度[N/rad]
49   % mu: 摩擦系数
50   % N_lf, N_rr: 左右驱动轮垂向载荷[N]
51   % W: 轮距[m]
52   % use_smooth: 是否使用光滑限幅
53   % smooth_gain: 光滑限幅增益

```

```

1  % rho: 空气密度[kg/m³]
2  % CdA: 风阻系数×面积[m²]
3  % 输出参数:
4  % ds: 状态导数向量 [X_dot Y_dot psi_dot v_dot omega_dot beta_dot]
5
6  % 设置默认参数值
7  if nargin < 22
8      use_smooth = false;
9      smooth_gain = 30.0;
10 end
11
12 % 分解状态向量
13 X = s(1); Y = s(2); psi = s(3); v = s(4); omega = s(5); beta = s(6);
14
15 % 动态计算空气阻力（基于当前瞬时速度）
16 F_aero = 0.5 * rho * CdA * v^2 * sign(v); % 空气阻力[N], 与速度平方成正比
17 F_drag = F_rolling_total + F_aero; % 总阻力 = 滚动阻力 + 空气阻力
18
19 % 重新计算轮胎侧偏角（基于当前状态和转向角）
20 alpha_f = (beta + Lf*omega/max(abs(v),low_speed_thresh)) - delta_lf;
21 alpha_r = (beta - Lr*omega/max(abs(v),low_speed_thresh)) - delta_rr;
22
23 % 线性侧偏力计算
24 Fy_f_lin = -C_af * alpha_f; % 前轮侧偏力[N]
25 Fy_r_lin = -C_ar * alpha_r; % 后轮侧偏力[N]
26
27 % 侧向力饱和限制（基于摩擦圆原理）
28 Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
29 Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]
30
31 % 应用侧向力限幅（可选择光滑或硬限幅）
32 if use_smooth
33     Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
34     Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
35 else
36     Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
37     Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
38 end
39
40 % 使用预计算的驱动力（避免重复限幅）
41 F_drive_actual = Fx_lf + Fx_rr; % 实际总驱动力[N]
42
43 % 驱动不对称产生的偏航力矩
44 Mz_drive = (W/2) * (Fx_lf - Fx_rr); % [Nm]
45
46 % 纵向动力学方程
47 a_x = (F_drive_actual - F_drag - F_slope) / max(m_eff_total,1e-6); % 纵向加速度[m/s²]
48
49 % 低速保护：防止倒车情况下的负加速度
50 if a_x < 0 && v < 0

```

```

1      v_dot = 0; % 速度为负时不允许进一步减速
2      else
3          v_dot = a_x; % 正常加速度
4      end
5
6      % 横摆动力学计算
7      Mz_tire = Fy_f*Lf - Fy_r*Lr; % 轮胎侧偏力产生的偏航力矩[Nm]
8      Mz_total = Mz_tire + Mz_drive; % 总偏航力矩 = 轮胎力矩 + 驱动力矩
9
10     % 恢复：横摆阻尼回到标称
11     C_damping = 1000.0; % 横摆阻尼系数 [Nm/(rad/s)]
12     Mz_damping = -C_damping * omega; % 阻尼力矩（与角速度成正比）[Nm]
13     Mz_total = Mz_total + Mz_damping; % 包含阻尼的总偏航力矩[Nm]
14
15     % 角加速度计算
16     omega_dot = Mz_total / max(I_z,1e-6); % [rad/s²]
17
18     % 角加速度限幅保护
19     omega_dot_limit = 5.0; % 最大角加速度限制 [rad/s²]
20     omega_dot = sat_sym(omega_dot, omega_dot_limit);
21
22     % 侧滑角动力学（添加强阻尼确保稳定性）
23     if abs(v) < low_speed_thresh
24         % 低速情况：使用强阻尼强制侧滑角收敛到零
25         beta_dot = -5.0*beta;
26     else
27         % 正常速度：基于力学原理的侧滑角动力学 + 强阻尼项
28         beta_dot = (Fy_f + Fy_r) / (m_eff_total * max(abs(v),low_speed_thresh/10)) - omega -
5.0*beta;
29     end
30
31     % 侧滑角变化率限幅
32     beta_dot_limit = deg2rad(10); % 限制侧滑角变化率[rad/s]
33     beta_dot = sat_sym(beta_dot, beta_dot_limit);
34
35     % 运动学方程
36     psi_dot = omega; % 航向角变化率[rad/s]
37     X_dot = v * cos(psi + beta); % X 方向速度[m/s]
38     Y_dot = v * sin(psi + beta); % Y 方向速度[m/s]
39
40     % 组装状态导数向量
41     ds = [X_dot; Y_dot; psi_dot; v_dot; omega_dot; beta_dot];
42
43     % 局部饱和函数定义
44     function y = sat(x, xmin, xmax)
45         y = min(max(x, xmin), xmax);
46     end
47 end
48
49 %% ===== 连续动力学核心函数（原版，保留兼容性） =====
50 function ds = continuous_dynamics_core(s, delta_lf, delta_rr, Fx_lf, Fx_rr, F_drag, F_slope, ...

```

```

1      m_eff_total, I_z, Lf, Lr, low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth,
smooth_gain)
2      % 连续动力学核心函数（原版）- 用于 RK4 积分，使用固定的阻力值
3      % 输入参数说明同 V2 版本，但 F_drag 为固定值而非动态计算
4
5      % 设置默认参数
6      if nargin < 20
7          use_smooth = false;
8          smooth_gain = 30.0;
9      end
10
11     % 状态分解
12     X = s(1); Y = s(2); psi = s(3); v = s(4); omega = s(5); beta = s(6);
13
14     % 侧偏角计算
15     alpha_f = delta_lf - (beta + Lf*omega/max(abs(v),1e-3));
16     alpha_r = delta_rr - (beta - Lr*omega/max(abs(v),1e-3));
17
18     % 侧向力计算与限幅
19     Fy_f_lin = -C_af * alpha_f;
20     Fy_r_lin = -C_ar * alpha_r;
21
22     Fy_f_max = mu * N_lf;
23     Fy_r_max = mu * N_rr;
24
25     if use_smooth
26         Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
27         Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
28     else
29         Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
30         Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
31     end
32
33     % 驱动力和偏航力矩
34     F_drive_actual = Fx_lf + Fx_rr;
35     Mz_drive = (W/2) * (Fx_lf - Fx_rr);
36
37     % 纵向动力学
38     a_x = (F_drive_actual - F_drag - F_slope) / max(m_eff_total,1e-6);
39
40     % 低速保护
41     if a_x < 0 && v < 0
42         v_dot = 0;
43     else
44         v_dot = a_x;
45     end
46
47     % 横摆动力学
48     Mz_tire = Fy_f*Lf - Fy_r*Lr;
49     Mz_total = Mz_tire + Mz_drive;
50

```



```

1   % 横摆阻尼
2   C_damping = 1000.0;
3   Mz_damping = -C_damping * omega;
4   Mz_total = Mz_total + Mz_damping;
5
6   omega_dot = Mz_total / max(I_z,1e-6);
7
8   % 角加速度限幅
9   omega_dot_limit = 5.0;
10  omega_dot = sat_sym(omega_dot, omega_dot_limit);
11
12  % 侧滑角动力学
13  if abs(v) < low_speed_thresh
14      beta_dot = -5.0*beta;
15  else
16      beta_dot = (Fy_f + Fy_r) / (m_eff_total * max(abs(v),low_speed_thresh/10)) - omega -
5.0*beta;
17  end
18
19  % 侧滑角变化率限幅
20  beta_dot_limit = deg2rad(10);
21  beta_dot = sat_sym(beta_dot, beta_dot_limit);
22
23  % 运动学
24  psi_dot = omega;
25  X_dot = v * cos(psi + beta);
26  Y_dot = v * sin(psi + beta);
27
28  % 输出状态导数
29  ds = [X_dot; Y_dot; psi_dot; v_dot; omega_dot; beta_dot];
30
31  % 局部饱和函数
32  function y = sat(x, xmin, xmax)
33      y = min(max(x, xmin), xmax);
34  end
35  end
36
37  %% ===== 工具函数集 =====
38  function y = sat(x, xmin, xmax)
39  % 标准饱和函数 - 将输入 x 限制在[xmin, xmax]范围内
40  % 输入: x-输入值, xmin-最小值, xmax-最大值
41  % 输出: y-饱和后的值
42  y = min(max(x, xmin), xmax);
43  end
44
45  function y = sat_sym(x, xmax)
46  % 对称饱和函数 - 将输入 x 限制在[-xmax, xmax]范围内
47  % 输入: x-输入值, xmax-对称的最大绝对值
48  % 输出: y-饱和后的值
49  y = min(max(x, -xmax), xmax);
50  end

```

```

1
2 function y = sat_smooth(x, xmin, xmax, k)
3 % 光滑饱和函数 - 使用双曲正切实现光滑限幅
4 % 输入: x-输入值, xmin-最小值, xmax-最大值, k-光滑增益 (越大越接近硬限幅)
5 % 输出: y-光滑饱和后的值
6 if nargin < 4
7     k = 30.0; % 默认光滑增益
8 end
9 if xmax <= xmin
10     y = (xmin + xmax) / 2; % 异常情况处理
11     return;
12 end
13 % 归一化处理 + 双曲正切光滑限幅
14 x_norm = (x - xmin) / (xmax - xmin); % 归一化到[0,1]
15 y_norm = (tanh(k * (x_norm - 0.5)) + 1) / 2; % 光滑 S 曲线变换
16 y = xmin + (xmax - xmin) * y_norm; % 反归一化
17 end
18
19 function y = softplus(x, k)
20 % 光滑的 max(x,0)近似函数 - 提高数值稳定性
21 % 输入: x-输入值, k-光滑增益 (越大越接近 max 函数)
22 % 输出: y-光滑处理后的值
23 if nargin < 2
24     k = 30.0; % 默认光滑增益
25 end
26 y = (1/k) * log(1 + exp(k * x)); % softplus 函数公式
27 end
28
29 function a = normalizeAngle(a)
30 % 角度归一化函数 - 将角度包装到[-pi, pi]范围内
31 % 输入: a-输入角度[rad]
32 % 输出: a-归一化后的角度[rad]
33 a = atan2(sin(a), cos(a)); % 使用 atan2 实现稳健的角度归一化
34 end
35
36 function [N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v,
theta_g, params)
37 % 载荷转移计算函数 - 基于纵向/横向加速度的简化载荷转移模型 + 坡度影响
38 % 输入参数:
39 %   F_cmd: 驱动力指令[N], omega: 横摆角速度[rad/s], v: 速度[m/s]
40 %   theta_g: 坡度角[rad], params: 参数结构体
41 % 输出参数:
42 %   N_lf, N_rf, N_lr, N_rr: 四轮垂向载荷[N]
43 %   F_rolling_total: 总滚动阻力[N]
44
45 % 提取参数
46 m = params.mass; % 质量[kg]
47 g = params.gravity; % 重力加速度[m/s²]
48 h_cg = params.h_cg; % 质心高度[m]
49 W = params.W; % 轮距[m]
50 L = params.L; % 轴距[m]

```

```

1  c_r = params.rolling_resistance;      % 滚动阻力系数
2  accel_limit = params.max_acceleration; % 最大加速度限制[m/s²]
3
4  % 纵向加速度估算（避免代数环）
5  F_rolling_est = c_r * m * g * cos(theta_g);      % 估算滚动阻力[N]
6  F_aero_est = 0.5 * 1.225 * 0.5 * v^2 * sign(v);   % 估算空气阻力[N]（简化）
7  F_slope_est = m * g * sin(theta_g);              % 坡度阻力[N]
8  F_net_est = F_cmd - F_rolling_est - F_aero_est - F_slope_est; % 净驱动力[N]
9  a_long = sat(F_net_est/m, -accel_limit, accel_limit); % 纵向加速度[m/s²]（限幅）
10
11 % 纵向载荷转移计算
12 Delta_long = m * a_long * (h_cg/L); % 前后轴载荷转移量[N]
13
14 % 横向加速度估算（基于当前运动状态）
15 % RF/LR 为万向支撑轮，只参与垂向支撑；侧向/偏航动力主要由 LF/RR 舵驱轮承担。
16 % 载荷转移需要左右转对称：幅值取 |v*omega|，方向单独由 sign(omega) 决定。
17 if abs(v) > 1e-3 && abs(omega) > 1e-6
18     a_lat = abs(v * omega);      % 横向向心加速度幅值[m/s²]
19     Delta_lat = m * a_lat * (h_cg/max(W,1e-6)); % 横向载荷转移量[N]
20 else
21     Delta_lat = 0; % 静止或极低速时无横向载荷转移
22 end
23
24 % 转向方向符号
25 sgn_turn = sign(omega);
26
27 % 基础轴载荷计算（考虑坡度影响）
28 W_total = m * g * cos(theta_g); % 总垂向载荷（坡度修正）[N]
29 N_front_base = W_total/2 - Delta_long/2; % 前轴基础载荷[N]
30 N_rear_base = W_total/2 + Delta_long/2; % 后轴基础载荷[N]
31
32 % 横向载荷分配到各轴
33 Delta_lat_front = sgn_turn * Delta_lat/2; % 前轴横向转移量[N]
34 Delta_lat_rear = sgn_turn * Delta_lat/2; % 后轴横向转移量[N]
35
36 % 四轮垂向载荷计算（确保非负）
37 N_lf = max(0, N_front_base/2 - Delta_lat_front/2); % 左前轮载荷[N]
38 N_rf = max(0, N_front_base/2 + Delta_lat_front/2); % 右前轮载荷[N]
39 N_lr = max(0, N_rear_base/2 - Delta_lat_rear/2); % 左后轮载荷[N]
40 N_rr = max(0, N_rear_base/2 + Delta_lat_rear/2); % 右后轮载荷[N]
41
42 % 基于实时载荷计算总滚动阻力
43 F_rolling_total = c_r * (N_lf + N_rf + N_lr + N_rr); % 总滚动阻力[N]
44 end
45
46 ===== FILE: src/core/output_eq.m =====
47 function y = output_eq(x, u, theta_ground, params)
48 % =====
49 % 文件名: output_eq.m
50 % 功能描述: 对角式双舵轮 AGV 输出方程

```

```

1 % 输入参数:
2 %   x: 状态向量 [X Y psi v omega delta_lf delta_rr beta] (8×1)
3 %   u: 输入向量 [F_cmd omega_cmd] (2×1)
4 %   theta_ground: 地面坡度角 (标量)
5 %   params: 参数结构体
6 % 输出参数:
7 %   y: 输出向量 (31×1) - V4.2 扩展版本
8 %       [1-8]   基本状态变量: X, Y, psi, v, omega, delta_lf, delta_rr, beta
9 %       [9-11]  IMU 测量: accel_x_meas, gyro_y_meas, gyro_z_meas
10 %      [12-13]  电流估计: I_meas_lf, I_meas_rr
11 %      [14-15]  扰动力估计: F_dist_calc_lf, F_dist_calc_rr
12 %      [16]     地面角度: theta_ground
13 %      [17-18]  轮速: omega_wheel_lf, omega_wheel_rr
14 %      [19-27]  AI 算法专用输出: 载荷信息、轮胎利用率、运动状态 (含前后侧偏角)
15 %      [28-31]  扩展诊断信息: slip_flag, stall_flag, N_lf, N_rr
16 % =====
17
18 %% 提取状态变量
19 X = x(1); Y = x(2); psi = x(3); v = x(4);
20 omega = x(5); delta_lf = x(6); delta_rr = x(7); beta = x(8);
21
22 %% 提取输入
23 F_cmd = u(1);
24 omega_cmd = u(2);
25
26 %% 直接获取参数 (从 parameters.m 结构体)
27 Ts = params.Ts;
28 m = params.mass;
29 I_z = params.Iz;
30 L = params.L;
31 W = params.W;
32 h_cg = params.h_cg;
33 r = params.wheel_radius;
34 n = params.gear_ratio;
35 eta = params.gear_efficiency;
36 k_t = params.motor_torque_constant;
37 mu = params.friction_coefficient;
38 c_r = params.rolling_resistance;
39 rho = params.air_density;
40 CdA = params.drag_coefficient_area;
41 C_af = params.front_cornering_stiffness;
42 C_ar = params.rear_cornering_stiffness;
43 delta_max = params.max_steering_angle;
44 delta_dot_max = params.max_steering_rate;
45 tau_delta = params.steering_time_constant;
46 current_limit = params.current_limit;
47 accel_limit = params.max_acceleration;
48 g = params.gravity;
49 min_omega_threshold = params.min_angular_velocity_threshold;
50 low_speed_thresh = params.low_speed_threshold;

```

```

1
2 % 噪声参数
3 enable_noise = params.enable_noise;
4 current_noise_std = params.current_noise_std;
5 wheel_speed_noise_std = params.wheel_speed_noise_std;
6 dist_noise_std = params.disturbance_noise_std;
7 noise_factor = double(enable_noise);
8
9 % 初始化随机数生成器（确保可复现，只在第一次调用时设置）
10 persistent seeded;
11 if enable_noise && params.random_seed > 0 && isempty(seeded)
12     rng(params.random_seed);
13     seeded = true;
14 end
15
16 %% === 几何：对角式转向目标（严格 ICR 约束，使用测量  $\omega$  抑制增益放大） ===
17 if abs(omega_cmd) < min_omega_threshold || v < low_speed_thresh
18     delta_lf_target = 0; delta_rr_target = 0;
19 else
20     % ICR 坐标（车辆坐标系）：用测量  $\omega$  替代命令  $\omega$ ，避免  $v \downarrow$  时目标舵角被放大
21     w_icr = omega; % 使用测量值
22     R_cmd = (v / max(abs(w_icr), 1e-6)) * sign(w_icr);
23     x_c = 0.0; y_c = R_cmd;
24     % 公式:  $\tan(\delta_{lf}) = (L/2 - x_c) / (y_c - W/2)$ ,  $\tan(\delta_{rr}) = -(x_c + L/2) / (y_c + W/2)$ 
25     delta_lf_target = atan2((L/2 - x_c), (y_c - W/2));
26     delta_rr_target = -atan2((x_c + L/2), (y_c + W/2));
27 end
28
29 % NOTE(open-loop test): 观测侧不更新/不预测转角，仅读取状态中的
30 % delta_lf/delta_rr。原“舵机一阶”计算对输出无影响，已移除以避免误读。
31
32 %% === 载荷转移 & 滚阻 ===
33 [N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_ground,
34 params);
35
36 %% === 电机/传动限制推算到车轮牵引上限 ===
37 F_wheel_max = current_limit * k_t * (eta * n) / max(r, 1e-6); % 单驱动轮
38 num_drive_wheels = 2; % lf & rr
39 F_cmd_max_total = num_drive_wheels * F_wheel_max;
40 F_cmd_eff = sat_sym(F_cmd, F_cmd_max_total);
41
42 %% === 侧偏角（简单自行车模型的近似，用  $\beta$  作为全车质心侧滑角） ===
43 % 轮侧偏角近似:  $\alpha_f = \delta_{lf} - (v_y + L_f \omega) / v_x \approx \delta_{lf} - (\beta + L_f \omega / v)$ 
44 %  $\alpha_r = -(\beta - L_r \omega / v)$ 
45 Lf = L/2; Lr = L/2; % 对称
46 v_x = max(v * cos(beta), 1e-3);
47 alpha_f = (beta + Lf*omega/max(v, 1e-3)) - delta_lf;
48 alpha_r = (beta - Lr*omega/max(v, 1e-3)) - delta_rr; % 对角式双舵轮，后轮也有转向
49
50 % 线性侧偏刚度 + 软饱和（限幅到  $\mu N$ ）
51 Fy_f_lin = -C_af * alpha_f;

```

```

1  Fy_r_lin = -C_ar * alpha_r;
2  Fy_f_max = mu * N_lf;
3  Fy_r_max = mu * N_rr;
4  % 侧向力限幅（支持光滑限幅）
5  use_smooth = params.use_smooth_saturation;
6  if use_smooth
7      smooth_gain = params.smooth_gain;
8      Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
9      Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
10 else
11 Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
12 Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
13 end
14
15 %% === 牵引力分配：按法向载荷比例 + 偏航控制 + 摩擦椭圆限幅 ===
16 % 修正：只在两个驱动轮（左前+右后）之间按载荷比例分配
17 W_drive = N_lf + N_rr; % 驱动轮总载荷（非全车总重）
18 w_lf = N_lf / max(W_drive, 1e-6); % 左前轮在驱动轮中的载荷占比
19 w_rr = N_rr / max(W_drive, 1e-6); % 右后轮在驱动轮中的载荷占比
20
21 % 直线/低曲率时等分扭矩，使观测与动力学一致
22 if (abs(omega_cmd) < min_omega_threshold) && (abs(omega) < min_omega_threshold)
23     w_lf = 0.5;
24     w_rr = 0.5;
25 end
26
27 % 恢复：观测侧与动力学保持一致的偏航力差估算（仅用于拆算分配，不闭环）
28 K_omega_p = 50.0;
29 e_omega = omega_cmd - omega;
30 omega_dot_desired = K_omega_p * e_omega;
31 omega_dot_max = 10.0;
32 omega_dot_desired = sat_sym(omega_dot_desired, omega_dot_max);
33 Mz_needed = I_z * omega_dot_desired;
34 Delta_Fx = 2 * Mz_needed / max(W, 1e-3);
35 Delta_Fx_max = max(abs(F_cmd_eff), 10) * 0.5;
36 Delta_Fx = sat_sym(Delta_Fx, Delta_Fx_max);
37
38 % 驱动力分配：基础（载荷比例）+ 偏航控制（力差）
39 Fx_lf_cmd = F_cmd_eff * w_lf + Delta_Fx/2;
40 Fx_rr_cmd = F_cmd_eff * w_rr - Delta_Fx/2;
41
42 % 摩擦椭圆：sqrt((Fx/μN)^2 + (Fy/μN)^2) <= 1
43 Fx_lf_allow = mu * N_lf * sqrt(max(1 - (Fy_f/max(mu*N_lf, 1e-6))^2, 0));
44 Fx_rr_allow = mu * N_rr * sqrt(max(1 - (Fy_r/max(mu*N_rr, 1e-6))^2, 0));
45 F_drive_lf = sign(Fx_lf_cmd) * min(abs(Fx_lf_cmd), Fx_lf_allow);
46 F_drive_rr = sign(Fx_rr_cmd) * min(abs(Fx_rr_cmd), Fx_rr_allow);
47
48 % 合力/合力矩
49 F_drive_total = F_drive_lf + F_drive_rr;
50

```

```

1  % 空气阻力 + 滚动阻力 + 坡度分量
2  F_aero = 0.5 * rho * CdA * v^2 * sign(v);
3  F_drag = F_rolling_total + F_aero;
4  F_slope = m * g * sin(theta_ground);
5
6  % 有效质量（加入旋转惯量等效）
7  wheel_inertia = params.wheel_inertia;
8  motor_inertia = params.motor_inertia;
9  m_eff_total = m + 2*(wheel_inertia + motor_inertia*n^2)/(r^2);
10
11  %% === 计算测量量 ===
12
13  % [9-11] IMU 测量（添加噪声）
14  accel_x_base = (F_drive_total - F_drag - F_slope) / max(m_eff_total,1e-6);
15  accel_x_meas = accel_x_base + noise_factor * 0.1 * randn;
16  gyro_y_meas = 0 + noise_factor * 0.01 * randn; % 俯仰角速度（添加小噪声）
17  gyro_z_meas = omega + noise_factor * 0.02 * randn;
18
19  % [17-18] 轮速（基于 ICR 几何/低速保护）
20  if abs(omega) > min_omega_threshold && abs(v) > 1e-4
21      R = v / omega; sgn = sign(omega); x_c = 0; y_c = R*sgn;
22      r_lf = sqrt((x_c - L/2)^2 + (y_c - W/2*sgn)^2);
23      r_rr = sqrt((x_c + L/2)^2 + (y_c + W/2*sgn)^2);
24      v_lf = (r_lf / max(abs(R),1e-6)) * v;
25      v_rr = (r_rr / max(abs(R),1e-6)) * v;
26  else
27      v_lf = v * cos(beta);
28      v_rr = v * cos(beta);
29  end
30  omega_wheel_lf = v_lf / max(r,1e-6) + noise_factor * wheel_speed_noise_std * randn;
31  omega_wheel_rr = v_rr / max(r,1e-6) + noise_factor * wheel_speed_noise_std * randn;
32
33  % [12-13] 电流估计（车轮侧力矩映射回电机电流）
34  I_meas_lf = F_drive_lf * r / max(n*eta*k_t,1e-6) + noise_factor * current_noise_std * randn;
35  I_meas_rr = F_drive_rr * r / max(n*eta*k_t,1e-6) + noise_factor * current_noise_std * randn;
36
37  % [14-15] 扰动观测（净外力残差的简单估计）：
38  F_inertia_total = m_eff_total * accel_x_meas;
39  W_total = m*g*cos(theta_ground); % 补充定义 W_total（全车重量，用于扰动估计分配）
40  w_ratio_lf = N_lf / max(W_total,1e-6);
41  w_ratio_rr = N_rr / max(W_total,1e-6);
42  F_inertia_lf = F_inertia_total * w_ratio_lf; F_inertia_rr = F_inertia_total * w_ratio_rr;
43  F_slope_lf = F_slope * w_ratio_lf; F_slope_rr = F_slope * w_ratio_rr;
44  F_rolling_lf = c_r * N_lf; F_rolling_rr = c_r * N_rr;
45  F_motor_actual_lf = F_drive_lf; F_motor_actual_rr = F_drive_rr;
46  F_dist_calc_lf = F_motor_actual_lf - F_rolling_lf - F_inertia_lf - F_slope_lf +
noise_factor*dist_noise_std*randn;
47  F_dist_calc_rr = F_motor_actual_rr - F_rolling_rr - F_inertia_rr - F_slope_rr +
noise_factor*dist_noise_std*randn;
48
49  %% 计算额外的 AI 算法输出变量
50  % 载荷分配信息

```

```

1  load_ratio_front = (N_lf + N_rf) / max(W_total, 1e-6);
2  load_ratio_rear = (N_lr + N_rr) / max(W_total, 1e-6);
3  load_transfer_lateral = abs(N_lf - N_rf) / max(N_lf + N_rf, 1e-6);
4
5  % 轮胎利用率（摩擦椭圆）
6  tire_utilization_lf = sqrt((F_drive_lf/max(mu*N_lf,1e-6))^2 + (Fy_f/max(mu*N_lf,1e-6))^2);
7  tire_utilization_rr = sqrt((F_drive_rr/max(mu*N_rr,1e-6))^2 + (Fy_r/max(mu*N_rr,1e-6))^2);
8
9  % 运动状态指标
10 lateral_accel = v * omega; % 横向加速度估计 [m/s^2] - 修正量纲错误
11 slip_angle_front = alpha_f;
12 slip_angle_rear = alpha_r;
13
14 % 驱动状态
15 drive_force_asymmetry = abs(F_drive_lf - F_drive_rr) / max(F_drive_lf + F_drive_rr, 1e-6);
16
17 % 二元状态标志
18 slip_flag = double(max(tire_utilization_lf, tire_utilization_rr) > 0.98); % 打滑标志
19 total_current = I_meas_lf + I_meas_rr;
20 motor_current_limit_total = 2 * current_limit;
21 stall_flag = double(v < low_speed_thresh && total_current > 0.8 * motor_current_limit_total); %
堵转标志
22
23 %% 汇总输出（31×1）- 为 AI 算法提供丰富信息
24 y = [
25     % [1-8] 基本状态变量
26     X; Y; psi; v; omega; delta_lf; delta_rr; beta;
27     % [9-11] IMU 测量
28     accel_x_meas; gyro_y_meas; gyro_z_meas;
29     % [12-13] 电流估计
30     I_meas_lf; I_meas_rr;
31     % [14-15] 扰动力估计
32     F_dist_calc_lf; F_dist_calc_rr;
33     % [16] 地面角度
34     theta_ground;
35     % [17-18] 轮速
36     omega_wheel_lf; omega_wheel_rr;
37     % [19-26] AI 算法专用输出
38     load_ratio_front; load_ratio_rear; load_transfer_lateral; % 载荷信息
39     tire_utilization_lf; tire_utilization_rr; % 轮胎利用率
40     lateral_accel; slip_angle_front; slip_angle_rear; drive_force_asymmetry; % 运动状态
41     % [28-31] 扩展诊断信息
42     slip_flag; stall_flag; % 二元状态标志
43     N_lf; N_rr % 法向载荷 (Fz)
44 ];
45
46 end
47
48 %% ===== 工具函数 =====
49 function y = sat(x, xmin, xmax)
50 y = min(max(x, xmin), xmax);

```



```

1   end
2
3   function y = sat_sym(x, xmax)
4   y = min(max(x, -xmax), xmax);
5   end
6
7   function y = sat_smooth(x, xmin, xmax, k)
8   % 光滑限幅函数:  $\text{sat}(x,a,b) \approx a + (b-a) * (\tanh(k*(x-a)/(b-a))+1)/2$ 
9   % k: 光滑增益, 越大越接近硬限幅
10  if nargin < 4
11      k = 30.0; % 默认增益
12  end
13  if xmax <= xmin
14      y = (xmin + xmax) / 2;
15      return;
16  end
17  x_norm = (x - xmin) / (xmax - xmin); % 归一化到[0,1]
18  y_norm = (tanh(k * (x_norm - 0.5)) + 1) / 2; % 光滑 S 曲线
19  y = xmin + (xmax - xmin) * y_norm;
20  end
21
22  function [N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v,
theta_g, params)
23  % 基于纵向/横向加速度的简化载荷转移 + 坡度影响
24  m = params.mass;
25  g = params.gravity;
26  h_cg = params.h_cg;
27  W = params.W;
28  L = params.L;
29  c_r = params.rolling_resistance;
30  accel_limit = params.max_acceleration;
31
32  % 纵向加速度 (使用指令近似, 以避免环依赖; 也可改为用上一拍估计)
33  a_long = sat(F_cmd/m, -accel_limit, accel_limit) - g*sin(theta_g);
34  Delta_long = m * a_long * (h_cg/L); % 前后轴载荷转移
35
36  % 横向加速度 (用 omega, v 估)
37  % RF/LR 为万向支撑轮, 只参与垂向支撑; 侧向/偏航动力主要由 LF/RR 舵驱轮承担。
38  % 载荷转移幅值取  $|v*\omega|$ , 方向由  $\text{sign}(\omega)$  单独决定, 避免右转时被夹到近零。
39  if abs(v) > 1e-3 && abs(omega) > 1e-6
40      a_lat = abs(v * omega);
41      Delta_lat = m * a_lat * (h_cg/max(W,1e-6));
42  else
43      Delta_lat = 0;
44  end
45  sgn_turn = sign(omega);
46
47  % 轴基础载荷 (坡度影响投影到总重)
48  W_total = m * g * cos(theta_g);
49  N_front_base = W_total/2 - Delta_long/2;
50  N_rear_base = W_total/2 + Delta_long/2;

```

```

1
2 % 横向分配到轴
3 Delta_lat_front = sgn_turn * Delta_lat/2;
4 Delta_lat_rear = sgn_turn * Delta_lat/2;
5
6 % 四轮法向载荷
7 N_lf = max(0, N_front_base/2 - Delta_lat_front/2);
8 N_rf = max(0, N_front_base/2 + Delta_lat_front/2);
9 N_lr = max(0, N_rear_base/2 - Delta_lat_rear/2);
10 N_rr = max(0, N_rear_base/2 + Delta_lat_rear/2);
11
12 % 滚动阻力（基于实时载荷）
13 F_rolling_total = c_r * (N_lf + N_rf + N_lr + N_rr);
14 end
15
16 ===== FILE: src/core/state_eq_ref.m =====
17 function x_next = state_eq_ref(x, u, theta_ground, params)
18 % =====
19 % 文件名: state_eq.m
20 % 功能描述: 对角式双舵轮 AGV 状态转移方程 - 实现车辆动力学模型的时间步进
21 % 输入参数:
22 %   x: 当前时刻状态向量 [X Y psi v omega delta_lf delta_rr beta] (8×1)
23 %       其中: X-全局 X 坐标[m], Y-全局 Y 坐标[m], psi-航向角[rad]
24 %           v-纵向速度[m/s], omega-横摆角速度[rad/s]
25 %           delta_lf-左前轮转向角[rad], delta_rr-右后轮转向角[rad]
26 %           beta-质心侧偏角[rad]
27 %   u: 控制输入向量 [F_cmd omega_cmd] (2×1)
28 %       F_cmd-总驱动力指令[N], omega_cmd-期望横摆角速度[rad/s]
29 %   theta_ground: 地面坡度角[rad] (标量)
30 %   params: 包含所有车辆参数的结构体
31 % 输出参数:
32 %   x_next: 下一时刻状态向量 (8×1)
33 % =====
34
35 %% 提取状态变量 - 从状态向量中分解各个状态分量
36 X = x(1); Y = x(2); psi = x(3); v = x(4);
37 omega = x(5); delta_lf = x(6); delta_rr = x(7); beta = x(8);
38
39 %% 提取控制输入 - 从输入向量分解控制指令
40 F_cmd = u(1); % 总驱动力指令[N]
41 omega_cmd = u(2); % 期望横摆角速度指令[rad/s]
42
43 %% 直接从参数结构体获取车辆参数 - 提高代码可读性和维护性
44 Ts = params.Ts; % 采样时间[s]
45 m = params.mass; % 车辆总质量[kg]
46 I_z = params.Iz; % 绕 Z 轴的转动惯量[kg·m²]
47 L = params.L; % 轴距[m]
48 W = params.W; % 轮距[m]
49 h_cg = params.h_cg; % 质心高度[m]
50 r = params.wheel_radius; % 车轮半径[m]

```

```

1  n = params.gear_ratio; % 传动比
2  eta = params.gear_efficiency; % 传动效率
3  k_t = params.motor_torque_constant; % 电机转矩常数[N·m/A]
4  mu = params.friction_coefficient; % 路面摩擦系数
5  c_r = params.rolling_resistance; % 滚动阻力系数
6  rho = params.air_density; % 空气密度[kg/m³]
7  CdA = params.drag_coefficient_area; % 风阻系数×迎风面积[m²]
8  C_af = params.front_cornering_stiffness; % 前轮侧偏刚度[N/rad]
9  C_ar = params.rear_cornering_stiffness; % 后轮侧偏刚度[N/rad]
10 delta_max = params.max_steering_angle; % 最大转向角限制[rad]
11 delta_dot_max = params.max_steering_rate; % 最大转向速率[rad/s]
12 tau_delta = params.steering_time_constant; % 转向系统时间常数[s]
13 current_limit = params.current_limit; % 电机电流限制[A]
14 accel_limit = params.max_acceleration; % 最大加速度限制[m/s²]
15 g = params.gravity; % 重力加速度[m/s²]
16 min_omega_threshold = params.min_angular_velocity_threshold; % 最小角速度阈值[rad/s]
17 low_speed_thresh = params.low_speed_threshold; % 低速阈值[m/s]
18
19 %% === 转向几何计算: 参考几何 ICR (|ω_ref|<阈值回退测量) ===
20 if abs(omega_cmd) < min_omega_threshold || v < low_speed_thresh
21     % 低速/小指令: 维持直行
22     delta_lf_target = 0; delta_rr_target = 0;
23 else
24     % 角速度误差抑制 (比例缩放, 保持两轮几何比不变)
25     e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]
26     if abs(omega) > abs(omega_cmd) * 1.2 && sign(omega) == sign(omega_cmd)
27         delta_scale = max(omega_cmd / max(abs(omega), 1e-6), 0.1);
28     else
29         delta_scale = 1.0;
30     end
31
32     % ICR: 优先用参考曲率 R_ref=v_ref/omega_ref; |ω_ref|<阈值或缺失时回退测量
33     w_ref_min = 1e-3; % 回退阈值 [rad/s]
34     try, ref = evalin('base','ref'); catch, ref = struct(); end
35     v_ref_use = NaN; omega_ref_use = NaN;
36     if isstruct(ref) && isfield(ref,'v_ref') && isfield(ref,'omega_ref')
37         if isfield(ref,'t') && ~isempty(ref.t)
38             try, t_sim = get_param(bdroot,'SimulationTime'); catch, t_sim = 0; end
39             Ts_loc = Ts;
40             idx = 1 + round(t_sim / max(Ts_loc,1e-6));
41             idx = max(1, min(idx, numel(ref.t)));
42             v_ref_use = ref.v_ref(idx);
43             omega_ref_use = ref.omega_ref(idx);
44         else
45             if isscalar(ref.v_ref), v_ref_use = ref.v_ref; end
46             if isscalar(ref.omega_ref), omega_ref_use = ref.omega_ref; end
47         end
48     end
49     use_ref = ~isnan(omega_ref_use) && abs(omega_ref_use) >= w_ref_min && ~isnan(v_ref_use);
50     if use_ref

```

```

1      R_cmd = (v_ref_use / max(abs(omega_ref_use),1e-6)) * sign(omega_ref_use);
2      else
3          w_icr = omega; % 回退：使用测量  $\omega$ 
4          R_cmd = (v / max(abs(w_icr),1e-6)) * sign(w_icr);
5      end
6      x_c = 0.0; % 对称转弯，ICR 在 y 轴上
7      y_c = R_cmd; % ICR y 坐标（可正可负）
8
9      % 使用带符号保护的分子，避免右转时象限翻转，同时保持右轮负号
10     denom_lf = y_c - W/2;
11     denom_rr = y_c + W/2;
12     safe_denom_lf = sign(denom_lf) * max(abs(denom_lf), 1e-6);
13     safe_denom_rr = sign(denom_rr) * max(abs(denom_rr), 1e-6);
14     delta_lf_geom = atan((L/2 - x_c) / safe_denom_lf);
15     delta_rr_geom = -atan((x_c + L/2) / safe_denom_rr);
16
17     % 误差缩放（不破坏两轮几何比例）
18     delta_lf_target = delta_lf_geom * delta_scale;
19     delta_rr_target = delta_rr_geom * delta_scale;
20
21     % 取消额外的“安全夹紧”：由执行器限幅 delta_max 与速率限幅保障
22 end
23
24 %% === 舵机动力学模拟：一阶惯性 + 速率限制 + 角度限制 ===
25 % 转向系统动力学：一阶惯性环节模拟转向执行器响应
26 delta_lf_dot = (delta_lf_target - delta_lf)/max(tau_delta,Ts);
27 delta_rr_dot = (delta_rr_target - delta_rr)/max(tau_delta,Ts);
28
29 % 转向速率限制 - 模拟转向执行器的物理限制
30 delta_lf_dot = sat(delta_lf_dot, -delta_dot_max, delta_dot_max);
31 delta_rr_dot = sat(delta_rr_dot, -delta_dot_max, delta_dot_max);
32
33 % 转向角更新 + 角度限制
34 delta_lf_new = sat(delta_lf + Ts*delta_lf_dot, -delta_max, delta_max);
35 delta_rr_new = sat(delta_rr + Ts*delta_rr_dot, -delta_max, delta_max);
36
37 %% === 载荷转移计算 & 滚动阻力计算 ===
38 [N_lf, N_rf, N_lr, N_rr, F_rolling_total] = compute_load_transfer(F_cmd, omega, v, theta_ground,
39 params);
40
41 %% === 电机/传动系统限制推算到车轮牵引力上限 ===
42 % 计算单个驱动轮的最大牵引力（基于电机电流限制）
43 F_wheel_max = current_limit * k_t * (eta * n) / max(r,1e-6); % 单驱动轮最大牵引力[N]
44
45 % 计算总驱动轮的最大牵引力（对角式：左前+右后两个驱动轮）
46 num_drive_wheels = 2; % 驱动轮数量：lf & rr
47 F_cmd_max_total = num_drive_wheels * F_wheel_max;
48
49 % 应用牵引力限制到指令值
50 F_cmd_eff = sat_sym(F_cmd, F_cmd_max_total);
51

```

```

1    %% === 轮胎侧偏角计算（基于简化自行车模型） ===
2    % 车辆几何参数：假设质心位于轴距中心
3    Lf = L/2; Lr = L/2; % 前后轴到质心的距离[m]
4
5    % 侧偏刚度：使用标称值（取消转弯增强的临时放大）
6    % C_af, C_ar 已由 parameters.m 提供
7
8    % 纵向速度分量（防止除零）
9    v_x = max(v * cos(beta), 1e-3);
10
11    % 前轮侧偏角（标准符号）：alpha_f = (beta + Lf*omega/v) - delta_lf
12    alpha_f = (beta + Lf*omega/max(v,low_speed_thresh)) - delta_lf_new;
13
14    % 后轮侧偏角（标准符号）：alpha_r = (beta - Lr*omega/v) - delta_rr
15    alpha_r = (beta - Lr*omega/max(v,low_speed_thresh)) - delta_rr_new; % 对角式双舵轮，后轮也有转向
16
17    % 线性侧偏力计算 + 软饱和（限幅到摩擦圆边界）
18    Fy_f_lin = -C_af * alpha_f; % 前轮线性侧偏力[N]
19    Fy_r_lin = -C_ar * alpha_r; % 后轮线性侧偏力[N]
20
21    % 基于垂向载荷计算最大侧向力（摩擦圆限制）
22    Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
23    Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]
24
25    % 应用侧向力饱和限制
26    Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
27    Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
28
29    %% === 牵引力分配策略：按法向载荷比例 + 偏航控制 ===
30    % 驱动轮总垂向载荷（仅考虑两个驱动轮：左前+右后）
31    W_drive = N_lf + N_rr; % 驱动轮总载荷[N]
32
33    % 计算各驱动轮在总驱动载荷中的权重比例
34    w_lf = N_lf / max(W_drive,1e-6); % 左前轮载荷占比
35    w_rr = N_rr / max(W_drive,1e-6); % 右后轮载荷占比
36
37    % 直线/低曲率时等分扭矩，抑制对角驱动导致的偏航力矩
38    if (abs(omega_cmd) < min_omega_threshold) && (abs(omega) < min_omega_threshold)
39        w_lf = 0.5;
40        w_rr = 0.5;
41    end
42
43    % 偏航控制：根据角速度误差产生驱动力差以实现横摆控制
44    e_omega = omega_cmd - omega; % 角速度跟踪误差[rad/s]
45
46    % 横摆角速度保护：防止角速度爆炸式增长
47    omega_max = 2.0; % 最大角速度限制 [rad/s]（远大于目标 0.1 rad/s）
48
49    if v < low_speed_thresh || abs(omega_cmd) < min_omega_threshold
50        % 低速/小指令：关断偏航力差，避免原地自旋

```

```

1      Delta_Fx = 0;
2  else
3      if abs(omega) > omega_max
4          % 情况：角速度超限 - 施加强制制动力矩进行保护
5          omega_sign = sign(omega);
6          Mz_brake = -omega_sign * I_z * 50.0; % 强制制动力矩[Nm]
7          Delta_Fx = 2 * Mz_brake / max(W, 1e-3); % 所需的驱动力差[N]
8      else
9          % 恢复：正常 P 控制产生偏航力差
10         K_omega_p = 120.0; % 偏航控制增益 (优化: 100~120)
11         omega_dot_desired = K_omega_p * e_omega; % 期望角加速度[rad/s²]
12         omega_dot_max = 10.0; % 最大角加速度 [rad/s²]
13         omega_dot_desired = sat_sym(omega_dot_desired, omega_dot_max);
14         Mz_needed = I_z * omega_dot_desired; % 所需偏航力矩 [Nm]
15         Delta_Fx = 2 * Mz_needed / max(W, 1e-3); % 所需驱动力差 [N]
16         % 解耦：按摩擦裕度设置 ΔFx_max (不再绑定 F_cmd_eff 大小)
17         Delta_Fx_max = 0.5 * mu * (N_lf + N_rr); % 驱动力差限幅 (保守系数 0.5)
18         Delta_Fx = sat_sym(Delta_Fx, Delta_Fx_max);
19     end
20 end
21
22 % 驱动力分配：基础分配 (按载荷比例) + 偏航控制 (力差调整)
23 Fx_lf_cmd = F_cmd_eff * w_lf + Delta_Fx/2; % 左前轮指令驱动力[N]
24 Fx_rr_cmd = F_cmd_eff * w_rr - Delta_Fx/2; % 右后轮指令驱动力[N]
25
26 %% === 摩擦椭圆限幅：确保合力不超过轮胎-路面摩擦极限 ===
27 % 获取是否使用光滑限幅的标志
28 use_smooth = params.use_smooth_saturation;
29
30 if use_smooth
31     % 光滑版本限幅：使用 softplus 函数替代 max(·,0)，提高数值稳定性
32     smooth_gain = params.smooth_gain;
33
34     % 计算各轮胎在摩擦椭圆中的可用纵向力余量
35     term_lf = softplus(1 - (Fy_f/max(mu*N_lf,1e-6))^2, smooth_gain);
36     term_rr = softplus(1 - (Fy_r/max(mu*N_rr,1e-6))^2, smooth_gain);
37
38     Fx_lf_allow = mu * N_lf * sqrt(term_lf); % 左前轮允许的最大纵向力[N]
39     Fx_rr_allow = mu * N_rr * sqrt(term_rr); % 右后轮允许的最大纵向力[N]
40
41     % 应用光滑限幅到指令驱动力
42     F_drive_lf = sat_smooth(Fx_lf_cmd, -Fx_lf_allow, Fx_lf_allow, smooth_gain);
43     F_drive_rr = sat_smooth(Fx_rr_cmd, -Fx_rr_allow, Fx_rr_allow, smooth_gain);
44 else
45     % 硬限幅版本：传统的摩擦椭圆限制
46     Fx_lf_allow = mu * N_lf * sqrt(max(1 - (Fy_f/max(mu*N_lf,1e-6))^2, 0));
47     Fx_rr_allow = mu * N_rr * sqrt(max(1 - (Fy_r/max(mu*N_rr,1e-6))^2, 0));
48
49     F_drive_lf = sign(Fx_lf_cmd) * min(abs(Fx_lf_cmd), Fx_lf_allow);
50     F_drive_rr = sign(Fx_rr_cmd) * min(abs(Fx_rr_cmd), Fx_rr_allow);

```

```

1   end
2
3   % 计算总有效驱动力
4   F_drive_total = F_drive_lf + F_drive_rr;
5
6   % 计算驱动不对称产生的偏航力矩（左右驱动力差异引起）
7   Mz_drive = (W/2) * (F_drive_lf - F_drive_rr);
8
9   % 坡度阻力计算
10  F_slope = m * g * sin(theta_ground);
11
12  % 有效质量计算（包含旋转部件的等效质量）
13  wheel_inertia = params.wheel_inertia;    % 车轮转动惯量[kg·m²]
14  motor_inertia = params.motor_inertia;    % 电机转动惯量[kg·m²]
15  m_eff_total = m + 2*(wheel_inertia + motor_inertia*n^2)/(r^2); % 总有效质量[kg]
16
17  %% === RK4 数值积分（转向角线性插值 + 动态空气阻力计算） ===
18  % 提取核心状态变量用于 RK4 积分
19  s_k = [X; Y; psi; v; omega; beta];
20
21  % 转向角中点值（用于 RK4 中间步骤的线性插值）
22  delta_lf_mid = (delta_lf + delta_lf_new)/2;
23  delta_rr_mid = (delta_rr + delta_rr_new)/2;
24
25  % 获取光滑限幅参数
26  use_smooth = params.use_smooth_saturation;
27  smooth_gain = params.smooth_gain;
28
29  % 定义连续动力学核心函数（闭包形式，传入当前计算的所有参数）
30  CONT_CORE = @(s, dl, dr) continuous_dynamics_core_v2(s, dl, dr, ...
31      F_drive_lf, F_drive_rr, F_rolling_total, F_slope, m_eff_total, I_z, Lf, Lr, ...
32      low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth, smooth_gain, ...
33      rho, CdA); % 传入空气阻力参数用于动态计算
34
35  % RK4 积分四步计算（使用转向角插值）
36  k1 = CONT_CORE(s_k, delta_lf, delta_rr); % 第一步：使用 k 时刻转向角
37  k2 = CONT_CORE(s_k + 0.5*Ts*k1, delta_lf_mid, delta_rr_mid); % 第二步：使用中点转向角
38  k3 = CONT_CORE(s_k + 0.5*Ts*k2, delta_lf_mid, delta_rr_mid); % 第三步：使用中点转向角
39  k4 = CONT_CORE(s_k + Ts*k3, delta_lf_new, delta_rr_new); % 第四步：使用 k+1 时刻转向角
40
41  % RK4 最终积分：加权平均
42  s_new = s_k + (Ts/6)*(k1 + 2*k2 + 2*k3 + k4);
43
44  %% 提取新状态并应用保护措施
45  X_new = s_new(1); Y_new = s_new(2); psi_new = s_new(3);
46  v_new = s_new(4); omega_new = s_new(5); beta_new = s_new(6);
47
48  % 航向角归一化到[-pi, pi]范围
49  psi_new = normalizeAngle(psi_new);
50

```

```

1    % 低速保护：防止速度变为负值（车辆不能倒车）
2    if v_new < 0
3        v_new = 0;
4    end
5
6    % 侧偏角限幅保护（防止数值爆炸）
7    beta_new = sat(beta_new, -deg2rad(15), deg2rad(15));
8
9    %% 组装下一时刻完整状态向量
10   x_next = [X_new; Y_new; psi_new; v_new; omega_new; delta_lf_new; delta_rr_new; beta_new];
11
12   end
13
14   %% ===== 连续动力学核心函数 V2（动态空气阻力计算）
=====
15   function ds = continuous_dynamics_core_v2(s, delta_lf, delta_rr, Fx_lf, Fx_rr, F_rolling_total,
F_slope, ...
16       m_eff_total, I_z, Lf, Lr, low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth,
smooth_gain, ...
17       rho, CdA)
18   % 连续动力学核心函数 V2 - 用于 RK4 积分的每个子步，动态计算空气阻力
19   % 输入参数：
20   %   s: 状态向量 [X Y psi v omega beta]
21   %   delta_lf, delta_rr: 当前转向角[rad]
22   %   Fx_lf, Fx_rr: 预计算的驱动力（已应用摩擦椭圆限幅）[N]
23   %   F_rolling_total: 总滚动阻力[N]
24   %   F_slope: 坡度阻力[N]
25   %   m_eff_total: 总有效质量[kg]
26   %   I_z: 转动惯量[kg·m²]
27   %   Lf, Lr: 前后轴到质心距离[m]
28   %   low_speed_thresh: 低速阈值[m/s]
29   %   C_af, C_ar: 前后轮侧偏刚度[N/rad]
30   %   mu: 摩擦系数
31   %   N_lf, N_rr: 左右驱动轮垂向载荷[N]
32   %   W: 轮距[m]
33   %   use_smooth: 是否使用光滑限幅
34   %   smooth_gain: 光滑限幅增益
35   %   rho: 空气密度[kg/m³]
36   %   CdA: 风阻系数×面积[m²]
37   % 输出参数：
38   %   ds: 状态导数向量 [X_dot Y_dot psi_dot v_dot omega_dot beta_dot]
39
40   % 设置默认参数值
41   if nargin < 22
42       use_smooth = false;
43       smooth_gain = 30.0;
44   end
45
46   % 分解状态向量
47   X = s(1); Y = s(2); psi = s(3); v = s(4); omega = s(5); beta = s(6);
48
49   % 动态计算空气阻力（基于当前瞬时速度）
50   F_aero = 0.5 * rho * CdA * v^2 * sign(v); % 空气阻力[N]，与速度平方成正比

```



```

1    F_drag = F_rolling_total + F_aero;          % 总阻力 = 滚动阻力 + 空气阻力
2
3    % 重新计算轮胎侧偏角（基于当前状态和转向角）
4    alpha_f = (beta + Lf*omega/max(abs(v),low_speed_thresh)) - delta_lf;
5    alpha_r = (beta - Lr*omega/max(abs(v),low_speed_thresh)) - delta_rr;
6
7    % 线性侧偏力计算
8    Fy_f_lin = -C_af * alpha_f; % 前轮侧偏力[N]
9    Fy_r_lin = -C_ar * alpha_r; % 后轮侧偏力[N]
10
11   % 侧向力饱和限制（基于摩擦圆原理）
12   Fy_f_max = mu * N_lf; % 前轮最大侧向力[N]
13   Fy_r_max = mu * N_rr; % 后轮最大侧向力[N]
14
15   % 应用侧向力限幅（可选择光滑或硬限幅）
16   if use_smooth
17       Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
18       Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
19   else
20       Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
21       Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
22   end
23
24   % 使用预计算的驱动力（避免重复限幅）
25   F_drive_actual = Fx_lf + Fx_rr; % 实际总驱动力[N]
26
27   % 驱动不对称产生的偏航力矩
28   Mz_drive = (W/2) * (Fx_lf - Fx_rr); % [Nm]
29
30   % 纵向动力学方程
31   a_x = (F_drive_actual - F_drag - F_slope) / max(m_eff_total,1e-6); % 纵向加速度[m/s²]
32
33   % 低速保护：防止倒车情况下的负加速度
34   if a_x < 0 && v < 0
35       v_dot = 0; % 速度为负时不允许进一步减速
36   else
37       v_dot = a_x; % 正常加速度
38   end
39
40   % 横摆动力学计算
41   Mz_tire = Fy_f*Lf - Fy_r*Lr; % 轮胎侧偏力产生的偏航力矩[Nm]
42   Mz_total = Mz_tire + Mz_drive; % 总偏航力矩 = 轮胎力矩 + 驱动力矩
43
44   % 调整：横摆阻尼减小以提高可转性
45   C_damping = 250.0; % 横摆阻尼系数 [Nm/(rad/s)]（优化：400→250）
46   Mz_damping = -C_damping * omega; % 阻尼力矩（与角速度成正比）[Nm]
47   Mz_total = Mz_total + Mz_damping; % 包含阻尼的总偏航力矩[Nm]
48
49   % 角加速度计算
50   omega_dot = Mz_total / max(I_z,1e-6); % [rad/s²]

```

```

1
2 % 角加速度限幅保护
3 omega_dot_limit = 5.0; % 最大角加速度限制 [rad/s²]
4 omega_dot = sat_sym(omega_dot, omega_dot_limit);
5
6 % 侧滑角动力学（添加强阻尼确保稳定性）
7 if abs(v) < low_speed_thresh
8     % 低速情况：使用强阻尼强制侧滑角收敛到零
9     beta_dot = -5.0*beta;
10 else
11     % 正常速度：基于力学原理的侧滑角动力学 + 强阻尼项
12     beta_dot = (Fy_f + Fy_r) / (m_eff_total * max(abs(v), low_speed_thresh/10)) - omega -
13     5.0*beta;
14 end
15 % 侧滑角变化率限幅
16 beta_dot_limit = deg2rad(10); % 限制侧滑角变化率[rad/s]
17 beta_dot = sat_sym(beta_dot, beta_dot_limit);
18
19 % 运动学方程
20 psi_dot = omega; % 航向角变化率[rad/s]
21 X_dot = v * cos(psi + beta); % X 方向速度[m/s]
22 Y_dot = v * sin(psi + beta); % Y 方向速度[m/s]
23
24 % 组装状态导数向量
25 ds = [X_dot; Y_dot; psi_dot; v_dot; omega_dot; beta_dot];
26
27 % 局部饱和函数定义
28 function y = sat(x, xmin, xmax)
29     y = min(max(x, xmin), xmax);
30 end
31 end
32
33 %% ===== 连续动力学核心函数（原版，保留兼容性） =====
34 function ds = continuous_dynamics_core(s, delta_lf, delta_rr, Fx_lf, Fx_rr, F_drag, F_slope, ...
35     m_eff_total, I_z, Lf, Lr, low_speed_thresh, C_af, C_ar, mu, N_lf, N_rr, W, use_smooth,
36     smooth_gain)
37 % 连续动力学核心函数（原版）- 用于 RK4 积分，使用固定的阻力值
38 % 输入参数说明同 V2 版本，但 F_drag 为固定值而非动态计算
39
40 % 设置默认参数
41 if nargin < 20
42     use_smooth = false;
43     smooth_gain = 30.0;
44 end
45
46 % 状态分解
47 X = s(1); Y = s(2); psi = s(3); v = s(4); omega = s(5); beta = s(6);
48
49 % 侧偏角计算
50 alpha_f = delta_lf - (beta + Lf*omega/max(abs(v), 1e-3));
51 alpha_r = delta_rr - (beta - Lr*omega/max(abs(v), 1e-3));

```

```

1
2 % 侧向力计算与限幅
3 Fy_f_lin = -C_af * alpha_f;
4 Fy_r_lin = -C_ar * alpha_r;
5
6 Fy_f_max = mu * N_lf;
7 Fy_r_max = mu * N_rr;
8
9 if use_smooth
10     Fy_f = sat_smooth(Fy_f_lin, -Fy_f_max, Fy_f_max, smooth_gain);
11     Fy_r = sat_smooth(Fy_r_lin, -Fy_r_max, Fy_r_max, smooth_gain);
12 else
13     Fy_f = sat(Fy_f_lin, -Fy_f_max, Fy_f_max);
14     Fy_r = sat(Fy_r_lin, -Fy_r_max, Fy_r_max);
15 end
16
17 % 驱动力和偏航力矩
18 F_drive_actual = Fx_lf + Fx_rr;
19 Mz_drive = (W/2) * (Fx_lf - Fx_rr);
20
21 % 纵向动力学
22 a_x = (F_drive_actual - F_drag - F_slope) / max(m_eff_total, 1e-6);
23
24 % 低速保护
25 if a_x < 0 && v < 0
26     v_dot = 0;
27 else
28     v_dot = a_x;
29 end
30
31 % 横摆动力学
32 Mz_tire = Fy_f*Lf - Fy_r*Lr;
33 Mz_total = Mz_tire + Mz_drive;
34
35 % 横摆阻尼
36 C_damping = 1000.0;
37 Mz_damping = -C_damping * omega;
38 Mz_total = Mz_total + Mz_damping;
39
40 omega_dot = Mz_total / max(I_z, 1e-6);
41
42 % 角加速度限幅
43 omega_dot_limit = 5.0;
44 omega_dot = sat_sym(omega_dot, omega_dot_limit);
45
46 % 侧滑角动力学
47 if abs(v) < low_speed_thresh
48     beta_dot = -5.0*beta;
49 else
50     beta_dot = (Fy_f + Fy_r) / (m_eff_total * max(abs(v), low_speed_thresh/10)) - omega -
5.0*beta;

```

```

1   fprintf('\n 评估完成, 关键指标: \n');
2   print_summary(summary);
3
4   end
5
6   %% =====
7   function rpt = evaluate_split(split, model, sample_limit)
8   % 针对单个数据集切分执行离线推理, 输出分类/回归指标
9   name = split.name;
10  X = split.X;
11  y_main = split.y_main;
12  y_turn = split.y_turn;
13  y_theta = split.y_theta;
14  mask_theta = split.mask_theta;
15  num_samples = size(X, 1);
16  if isfinite(sample_limit)
17      num_samples = min(num_samples, sample_limit);
18  end
19  fprintf(' [%s] 样本数: %d\n', name, num_samples);
20
21  pred_main = zeros(num_samples, 1);
22  pred_turn = zeros(num_samples, 1);
23  pred_theta = zeros(num_samples, 1);
24  conf_main = zeros(num_samples, 1);
25
26  for i = 1:num_samples
27      seq = squeeze(X(i, :, :));
28      % 单条序列送入 GRU_infer, 获得三任务输出与置信度
29      [label_main, label_turn, theta_hat, conf] = GRU_infer(seq, model);
30      pred_main(i) = label_main;
31      pred_turn(i) = label_turn;
32      pred_theta(i) = theta_hat;
33      conf_main(i) = max(conf.conf_main);
34  end
35
36  metrics_main = classification_metrics(y_main(1:num_samples), pred_main,
{'flat', 'stall', 'slope'});
37  metrics_turn = classification_metrics(y_turn(1:num_samples) + 2, pred_turn + 2,
{'right', 'straight', 'left'});
38
39  % 坡度 MAE 仅对 slope 样本
40  theta_idx = find(mask_theta(1:num_samples) > 0);
41  if isempty(theta_idx)
42      mae_theta = NaN;
43  else
44      mae_theta = mean(abs(pred_theta(theta_idx) - y_theta(theta_idx)));
45  end
46
47  rpt = struct();
48  rpt.name = name;
49  rpt.num_samples = num_samples;
50  rpt.metrics_main = metrics_main;

```

```

1  rpt.metrics_turn = metrics_turn;
2  rpt.mae_theta = mae_theta;
3  rpt.avg_confidence = mean(conf_main);
4
5  fprintf('    主分类准确率: %.2f%% | MAE_theta: %.3f deg\n', metrics_main.accuracy*100,
rad2deg(mae_theta));
6  end
7
8  %% =====
9  function rpt = evaluate_run(run_data, model, params, seq_len, enable_plots, results_dir)
10 % 真实 run 数据评估: 调用在线分类器, 统计准确率/延迟并可视化
11 state = GRU_state_classifier('init', params, model);
12 N = size(run_data.y_raw, 1);
13 out_main = zeros(N,1);
14 out_turn = zeros(N,1);
15 out_theta = zeros(N,1);
16 for k = 1:N
17     y = run_data.y_raw(k, :);
18     % 逐样本更新在线分类器内部状态
19     [state, out] = GRU_state_classifier('update', state, y);
20     out_main(k) = out.label_main;
21     out_turn(k) = out.label_turn;
22     out_theta(k) = out.theta_hat;
23 end
24
25 truth_main = run_data.label_main;
26 truth_turn = run_data.label_turn;
27 truth_theta = run_data.theta;
28 time = run_data.t;
29 start_idx = min(N, seq_len + 1);
30
31 rpt = struct();
32 rpt.scene = get_scene_name(run_data);
33 rpt.duration = time(end);
34 rpt.acc_main = mean(out_main(start_idx:end) == truth_main(start_idx:end));
35 rpt.acc_turn = mean(out_turn(start_idx:end) == truth_turn(start_idx:end));
36 slope_mask = truth_main(start_idx:end) == 3;
37 if any(slope_mask)
38     slope_idx = find(slope_mask) + start_idx - 1;
39     rpt.mae_theta = mean(abs(out_theta(slope_idx) - truth_theta(slope_idx)));
40 else
41     rpt.mae_theta = NaN;
42 end
43
44 % 检测延迟: 真值首次进入 slope 与预测首次进入 slope 的时间差
45 truth_slope_idx = find(truth_main == 3, 1, 'first');
46 pred_slope_idx = find(out_main == 3, 1, 'first');
47 if ~isempty(truth_slope_idx) && ~isempty(pred_slope_idx)
48     rpt.slope_delay = time(pred_slope_idx) - time(truth_slope_idx);
49 else
50     rpt.slope_delay = NaN;

```

```

1   end
2
3   if enable_plots
4       fig = figure('Name', sprintf('GRU 在线评估-%s', rpt.scene), 'Position', [100 100 1200 800]);
5       tiledlayout(3,1,'Padding','compact');
6       % 主分类对比
7       nexttile; hold on; grid on;
8       plot(time, truth_main, 'k','LineWidth',2);
9       plot(time, out_main, 'r--','LineWidth',1.2);
10      yticks(1:3); yticklabels({'flat','stall','slope'});
11      ylabel('主分类'); title(sprintf('%s | 主分类准确率 %.2f%%', rpt.scene, rpt.acc_main*100));
12      % 转弯分类对比
13      nexttile; hold on; grid on;
14      plot(time, truth_turn, 'k','LineWidth',2);
15      plot(time, out_turn, 'b--','LineWidth',1.2);
16      yticks(-1:1); yticklabels({'right','straight','left'});
17      ylabel('转弯'); title(sprintf('转弯准确率 %.2f%%', rpt.acc_turn*100));
18      % 坡度角估计曲线
19      nexttile; hold on; grid on;
20      plot(time, rad2deg(truth_theta), 'k','LineWidth',2);
21      plot(time, rad2deg(out_theta), 'g--','LineWidth',1.2);
22      ylabel('\theta [deg]'); xlabel('时间 [s]');
23      title(sprintf('坡度估计 MAE %.3f deg | 延迟 %.2f s', rad2deg(rpt.mae_theta),
rpt.slope_delay));
24      saveas(fig, fullfile(results_dir, sprintf('online_eval_%s.png', rpt.scene)));
25      close(fig);
26  end
27
28  end
29
30  %% =====
31  function metrics = classification_metrics(y_true, y_pred, class_names)
32  % 统一计算准确率、精确率、召回率、F1 以及混淆矩阵
33  cm = confusionmat(y_true(:), y_pred(:));
34  num_classes = size(cm,1);
35  precision = zeros(num_classes,1);
36  recall = zeros(num_classes,1);
37  f1 = zeros(num_classes,1);
38  for c = 1:num_classes
39      tp = cm(c,c);
40      fp = sum(cm(:,c)) - tp;
41      fn = sum(cm(c,:)) - tp;
42      precision(c) = tp / max(tp + fp, eps);
43      recall(c) = tp / max(tp + fn, eps);
44      f1(c) = 2 * precision(c) * recall(c) / max(precision(c) + recall(c), eps);
45  end
46  metrics = struct();
47  metrics.accuracy = sum(diag(cm)) / sum(cm(:));
48  metrics.confusion = cm;
49  metrics.precision = precision;
50  metrics.recall = recall;

```

```

1  metrics.f1 = f1;
2  metrics.class_names = class_names(:)';
3  end
4
5  %% =====
6  function summary = build_summary(split_reports, online_reports, online_by_mode, seq_len)
7  % 聚合离线/在线指标, 用于最终对比展示
8  summary = struct();
9  summary.seq_len = seq_len;
10 summary.offline = split_reports;
11 summary.online = online_reports;
12 summary.online_by_mode = online_by_mode;
13 summary.mean_acc_main = mean(cellfun(@(r) r.metrics_main.accuracy, split_reports));
14 summary.mean_mae_theta = mean(cellfun(@(r) r.mae_theta, split_reports));
15 if isempty(online_reports)
16     summary.mean_online_delay = NaN;
17 else
18     summary.mean_online_delay = mean(cellfun(@(r) r.slope_delay, online_reports), 'omitnan');
19 end
20 end
21
22 %% =====
23 function print_summary(summary)
24 % 终端输出核心指标, 方便快速查看
25 fprintf(' - 平均主分类准确率 (Train/Val/Test): %.2f%%\n', summary.mean_acc_main*100);
26 fprintf(' - 平均离线坡度 MAE: %.3f deg\n', rad2deg(summary.mean_mae_theta));
27 if isnan(summary.mean_online_delay)
28     fprintf(' - 平均在线坡度识别延迟: NaN (本次未选择到含坡度的 run) (seq_len=%.0f)\n',
summary.seq_len);
29 else
30     fprintf(' - 平均在线坡度识别延迟: %.2f s (seq_len=%.0f)\n', summary.mean_online_delay,
summary.seq_len);
31 end
32 print_mode_breakdown(summary.online_by_mode);
33 end
34
35 %% =====
36 function print_mode_breakdown(mode_entries)
37 % 输出按行驶模式分组的在线评估结果
38 if isempty(mode_entries)
39     fprintf(' - 未选择任何行驶模式样本, 跳过在线分组统计.\n');
40     return;
41 end
42
43 fprintf('\n 行驶模式分组指标: \n');
44 for i = 1:numel(mode_entries)
45     reports_cell = mode_entries(i).reports;
46     if isempty(reports_cell)
47         continue;
48     end
49     reports = [reports_cell{:}];
50     acc_main = mean(arrayfun(@(r) r.acc_main, reports));

```

```

1      acc_turn = mean(arrayfun(@(r) r.acc_turn, reports));
2      mae_theta = mean(arrayfun(@(r) r.mae_theta, reports), 'omitnan');
3      delay = mean(arrayfun(@(r) r.slope_delay, reports), 'omitnan');
4      fprintf('    - %-15s | 样本:%2d | 主分类:%.2f%% | 转弯:%.2f%% |  $\theta$  MAE:%.3f deg | 延迟:%s\n', ...
5              mode_entries(i).mode, numel(reports), acc_main*100, acc_turn*100, rad2deg(mae_theta),
6              format_delay(delay));
7      end
8      end
9      function txt = format_delay(delay)
10     if isnan(delay)
11         txt = 'NaN';
12     else
13         txt = sprintf('%.2f s', delay);
14     end
15     end
16
17     %% =====
18     function name = get_scene_name(source)
19     % 从 run/元数据中解析场景名称，兼容嵌套结构与不同字段命名
20     default_name = 'run_unknown';
21     name = default_name;
22
23     if nargin == 0 || isempty(source)
24         return;
25     end
26
27     % 直接传入字符串时立即返回
28     direct_text = extract_scene_text(source);
29     if ~isempty(direct_text)
30         name = direct_text;
31         return;
32     end
33
34     if ~isstruct(source)
35         return;
36     end
37
38     text_fields = {'scene', 'path_type', 'scene_name', 'mode'};
39     for i = 1:numel(text_fields)
40         field = text_fields{i};
41         if isfield(source, field)
42             candidate = extract_scene_text(source.(field));
43             if ~isempty(candidate)
44                 name = candidate;
45                 return;
46             end
47         end
48     end
49
50     % 递归查找常见嵌套字段

```



```

1  nested_fields = {'meta','path_params'};
2  for i = 1:numel(nested_fields)
3      field = nested_fields{i};
4      if isfield(source, field)
5          candidate = get_scene_name(source.(field));
6          if ~strcmp(candidate, default_name)
7              name = candidate;
8              return;
9          end
10     end
11 end
12
13 end
14
15 function text = extract_scene_text(value)
16 % 统一提取字符串表示, 兼容 char/string/cellstr
17 text = '';
18 if ischar(value)
19     text = strtrim(value);
20 elseif isstring(value) && isscalar(value)
21     text = strtrim(char(value));
22 elseif iscell(value) && numel(value) == 1
23     text = extract_scene_text(value{1});
24 end
25
26 if isstring(text)
27     text = char(text);
28 end
29
30 if isempty(text)
31     text = '';
32 end
33 end
34
35 %% =====
36 function cfg = apply_defaults(cfg, defaults)
37 % 递归填充缺省配置项, 保持主逻辑简洁
38 fields = fieldnames(defaults);
39 for i = 1:numel(fields)
40     f = fields{i};
41     if ~isfield(cfg, f) || isempty(cfg.(f))
42         cfg.(f) = defaults.(f);
43     end
44 end
45 end
46
47 ===== FILE: src/tests/test_gru_latency.m =====
48 % =====
49 % 文件名: test_gru_latency.m
50 % 版本号: V1.1

```

```

1  % 最后修改时间: 2025-11-26
2  % 作者: LPV-MPC Project
3  % 功能描述:
4  %   使用控制变量法 + 预设组合验证, 对 GRU 在线识别中的
5  %   dwell_main / dwell_turn / tau_theta 三个延时敏感参数进行搜索,
6  %   输出综合评分排名与最优配置。
7  %
8  % 配置说明:
9  %   - cfg.model_file       : GRU 模型文件 (默认 GRU_model.mat)
10 %   - cfg.raw_data_file    : 原始 run 数据 (默认 GRU_train_data_full.mat)
11 %   - cfg.results_dir      : 评估结果输出目录
12 %   - cfg.runs_per_mode    : 每种行驶模式抽取的 run 数量
13 %   - cfg.weights          : 综合得分权重 (准确率/延时/θMAE/抖动)
14 %
15 % 依赖脚本/数据:
16 %   - parameters.m (系统参数)
17 %   - GRU_model.mat (包含 seq_len/scaler 的训练模型)
18 %   - GRU_train_data_full.mat (在线评估所需 run)
19 %   - GRU_state_classifier.m / GRU_infer.m (在线推理封装)
20 %
21 % 使用方法:
22 %   >> test_gru_latency
23 %   运行结束后, Top5 排行及控制变量法最优结果将打印至命令行,
24 %   并保存至 cfg.results_dir/latency_summary_<timestamp>.mat。
25 %
26 % 约束/注意:
27 %   - 仅覆盖 GRU_state_classifier 中的驻留时间与 theta 低通参数;
28 %   - seq_len、特征滤波 (tau_diff/tau_accel_lp) 与数据集保持不变;
29 %   - 若需评估其它参数, 应另行构建专用脚本。
30 % =====
31
32 clear; clc; % 清理环境, 以免旧变量干扰评分
33
34 root = project_root();
35 data_models_dir = fullfile(root, 'data', 'models');
36 data_gru_dir = fullfile(root, 'data', 'gru');
37 default_results_dir = results_dir('gru/latency_eval');
38 fprintf('\n=====');
39 fprintf('GRU 延时评估 (test_gru_latency)\n');
40 fprintf('=====');
41
42 %% 配置
43 cfg = struct();
44 cfg.model_file = fullfile(data_models_dir, 'GRU_model.mat');
45 cfg.raw_data_file = fullfile(data_gru_dir, 'GRU_train_data_full.mat');
46 cfg.results_dir = default_results_dir;
47 cfg.runs_per_mode = 6; % 每种行驶模式最多选取的 run 数
48 cfg.weights = struct( ...
49     'acc_main', 1.0, ... % 主分类准确率权重
50     'acc_turn', 0.5, ... % 转弯准确率权重

```

```

1      'delay',    0.7, ...           % 延时惩罚系数 (单位 s)
2      'theta',    0.2, ...           %  $\theta$  MAE 惩罚系数 (单位 deg)
3      'flips',    0.3 ...            % 标签抖动惩罚系数 (次/分钟)
4  );
5
6  if ~exist(cfg.results_dir,'dir')
7      mkdir(cfg.results_dir);
8  end
9
10 %% 加载依赖与原始数据
11 fprintf('[1/6] 加载模型与数据...\n');
12 params = parameters();
13 model_data = load(cfg.model_file, 'model');
14 model = model_data.model;
15 raw = load(cfg.raw_data_file, 'data');
16 all_runs = raw.data.runs;
17 [selected_runs, selection_meta] = select_runs_for_latency(all_runs, cfg.runs_per_mode);
18 if isempty(selected_runs)
19     error('未选到任何 run, 请检查原始数据。');
20 end
21 fprintf(' - 选取 %d 条 run, 覆盖 %d 种行驶模式\n', numel(selected_runs),
selection_meta.num_modes);
22
23 timestamp = datestr(now,'yyyymmdd_HHMMSS');
24 variant_cache = containers.Map(); % key -> metrics
25 all_metrics = containers.Map();
26
27 %% 变体定义 (包含控制变量序列与人工组合)
28 % 最优组合 (来自上一轮评估):
29 %   dwell_main = 0.20 s
30 %   dwell_turn = 0.40 s
31 %   tau_theta = 0.15 s
32 baseline = struct('name','baseline','dwell_main',0.20,'dwell_turn',0.40,'tau_theta',0.15);
33 dwell_main_candidates = [0.20 0.25 0.30 0.40];
34 dwell_turn_candidates = [0.40 0.50];
35 tau_theta_candidates = [0.15 0.20 0.40];
36 manual_combos = [ ...
37     struct('name','combo_fast','dwell_main',0.25,'dwell_turn',0.40,'tau_theta',0.20); ...
38     struct('name','combo_balanced','dwell_main',0.30,'dwell_turn',0.50,'tau_theta',0.20); ...
39     struct('name','combo_aggressive','dwell_main',0.20,'dwell_turn',0.40,'tau_theta',0.15) ...
40 ];
41
42 %% 基线评估: 记录默认参数的表现
43 fprintf('\n[2/6] 评估基线参数...\n');
44 [metrics, variant_cache] = eval_variant_with_cache(baseline, params, model, selected_runs, cfg,
variant_cache);
45 all_metrics(metrics.key) = metrics;
46 control_variant = baseline;
47 control_metrics = metrics;
48 fprintf(' baseline score = %.3f (AccMain=%.3f, Delay=%.3f s)\n', control_metrics.score,
control_metrics.acc_main, control_metrics.delay_combined);
49
50 %% 控制变量法: 仅改变 dwell_main, 锁定其余参数

```

```

1    fprintf('\n[3/6] 控制变量法 - 扫描 dwell_main ...\n');
2    [control_variant, control_metrics, variant_cache, all_metrics] =
sweep_parameter('dwell_main', ...
3        dwell_main_candidates, control_variant, params, model, selected_runs, cfg, variant_cache,
all_metrics);
4    fprintf('  best dwell_main=%.2f s (score=%.3f)\n', control_variant.dwell_main,
control_metrics.score);
5
6    %% 控制变量法: 仅改变 dwell_turn
7    fprintf('\n[4/6] 控制变量法 - 扫描 dwell_turn ...\n');
8    [control_variant, control_metrics, variant_cache, all_metrics] =
sweep_parameter('dwell_turn', ...
9        dwell_turn_candidates, control_variant, params, model, selected_runs, cfg, variant_cache,
all_metrics);
10   fprintf('  best dwell_turn=%.2f s (score=%.3f)\n', control_variant.dwell_turn,
control_metrics.score);
11
12   %% 控制变量法: 仅改变 tau_theta
13   fprintf('\n[5/6] 控制变量法 - 扫描 tau_theta ...\n');
14   [control_variant, control_metrics, variant_cache, all_metrics] = sweep_parameter('tau_theta', ...
15       tau_theta_candidates, control_variant, params, model, selected_runs, cfg, variant_cache,
all_metrics);
16   fprintf('  best tau_theta=%.2f s (score=%.3f)\n', control_variant.tau_theta,
control_metrics.score);
17   control_best = control_metrics;
18
19   %% 手工组合验证: 捕捉潜在参数耦合
20   fprintf('\n[6/6] 评估手工组合...\n');
21   for i = 1:numel(manual_combos)
22       combo = manual_combos(i);
23       [combo_metrics, variant_cache] = eval_variant_with_cache(combo, params, model, selected_runs,
cfg, variant_cache);
24       combo_metrics.tag = sprintf('manual:%s', combo.name);
25       all_metrics(combo_metrics.key) = combo_metrics;
26   end
27
28   %% 汇总排行榜并输出 Top5
29   scoreboard = collect_metrics(all_metrics);
30   [~, order] = sort([scoreboard.score], 'descend');
31   scoreboard = scoreboard(order);
32
33   fprintf('\n===== \n');
34   fprintf('评估完成, Top 5 方案如下 (score 越大越好): \n');
35   fprintf('%-16s %-7s %-7s %-7s | Score  AccMain AccTurn  Delay  ΘMAE  Flips/min\n', 'Variant',
'd_main', 'd_turn', 'tauθ');
36   fprintf('%s\n', repmat('-',1,86));
37   for i = 1:min(5, numel(scoreboard))
38       m = scoreboard(i);
39       fprintf('%-16s %-7.2f %-7.2f %-7.2f | %5.3f  %5.3f  %5.3f  %5.3f  %5.3f  %5.2f\n', ...
40           m.tag, m.variant.dwell_main, m.variant.dwell_turn, m.variant.tau_theta, ...
41           m.score, m.acc_main, m.acc_turn, m.delay_combined, m.theta_mae_deg, m.flip_rate_per_min);
42   end
43
44   fprintf('\n 控制变量法最优: %s (score=%.3f)\n', control_best.tag, control_best.score);
45   fprintf('    - dwell_main=%.2f s, dwell_turn=%.2f s, tau_theta=%.2f s\n', ...
46       control_best.variant.dwell_main, control_best.variant.dwell_turn,
control_best.variant.tau_theta);
47
48   summary_file = fullfile(cfg.results_dir, sprintf('latency_summary_%s.mat', timestamp));
49   save(summary_file, 'scoreboard', 'cfg', 'selection_meta', 'control_best');
50   fprintf('\n 已保存评估结果至 %s\n', summary_file);

```

```

1
2 %% 辅助函数
3 function [metrics, cache] = eval_variant_with_cache(variant, params, model, runs, cfg, cache)
4     % 带缓存的变体评估, 避免重复仿真相同组合
5     key = variant_key(variant);
6     if isKey(cache, key)
7         metrics = cache(key);
8         return;
9     end
10    run_stats = arrayfun(@(r) simulate_and_score_run(r, params, model, variant), runs);
11    metrics = aggregate_metrics(run_stats, variant, cfg);
12    metrics.key = key;
13    metrics.tag = variant.name;
14    cache(key) = metrics;
15 end
16
17 function [best_variant, best_metrics, cache, metrics_map] = sweep_parameter(param_name, values,
base_variant, params, model, runs, cfg, cache, metrics_map)
18     % 对单个参数做一维扫描, 返回得分最高的组合
19     best_variant = base_variant;
20     best_metrics = [];
21     best_score = -inf;
22     for v = values
23         variant = base_variant;
24         variant.(param_name) = v;
25         variant.name = sprintf('%s_%s%.2f', base_variant.name, param_name, v);
26         [metrics, cache] = eval_variant_with_cache(variant, params, model, runs, cfg, cache);
27         metrics.tag = sprintf('sweep:%s', param_name);
28         metrics_map(metrics.key) = metrics;
29         if metrics.score > best_score
30             best_score = metrics.score;
31             best_variant = variant;
32             best_metrics = metrics;
33         end
34     end
35 end
36
37 function stats = simulate_and_score_run(run, params, model, variant)
38     % 针对单条 run 调用在线推理, 采集预测轨迹并计算指标
39     state = GRU_state_classifier('init', params, model);
40     state = apply_variant_overrides(state, variant);
41     seq_len_effective = state.seq_len;
42     N = size(run.y_raw,1);
43     pred_main = zeros(N,1);
44     pred_turn = zeros(N,1);
45     pred_theta = zeros(N,1);
46     for k = 1:N
47         y = run.y_raw(k,:);
48         [state, out] = GRU_state_classifier('update', state, y);
49         pred_main(k) = out.label_main;
50         pred_turn(k) = out.label_turn;

```

```

1      pred_theta(k) = out.theta_hat;
2      end
3      stats.seq_len = seq_len_effective;
4      stats.time = run.t;
5      stats.truth_main = run.label_main;
6      stats.truth_turn = run.label_turn;
7      stats.truth_theta = run.theta;
8      stats.pred_main = pred_main;
9      stats.pred_turn = pred_turn;
10     stats.pred_theta = pred_theta;
11     stats = compute_run_metrics(stats);
12 end
13
14 function state = apply_variant_overrides(state, variant)
15     % 将变体中的 dwell_main/dwell_turn/tau_theta 覆盖到 state
16     if isfield(variant, 'dwell_main')
17         state.dwell_main = variant.dwell_main;
18         state.dwell_main_steps = max(1, round(state.dwell_main / state.Ts));
19     end
20     if isfield(variant, 'dwell_turn')
21         state.dwell_turn = variant.dwell_turn;
22         state.dwell_turn_steps = max(1, round(state.dwell_turn / state.Ts));
23     end
24     if isfield(variant, 'tau_theta')
25         state.tau_theta = variant.tau_theta;
26         state.alpha_theta = state.Ts / (state.tau_theta + state.Ts);
27     end
28 end
29
30 function stats = compute_run_metrics(sim)
31     % 针对单条 run 的预测与真值, 计算准确率、延时、抖动等指标
32     time = sim.time;
33     N = numel(time);
34     start_idx = min(N, sim.seq_len + 1);
35     idx = start_idx:N;
36     if isempty(idx)
37         idx = 1:N;
38     end
39     duration = time(end) - time(idx(1));
40     stats.acc_main = mean(sim.pred_main(idx) == sim.truth_main(idx));
41     stats.acc_turn = mean(sim.pred_turn(idx) == sim.truth_turn(idx));
42     slope_mask = (sim.truth_main(idx) == 4);
43     if any(slope_mask)
44         stats.theta_mae = mean(abs(sim.pred_theta(idx(slope_mask)) -
sim.truth_theta(idx(slope_mask))));
45     else
46         stats.theta_mae = NaN;
47     end
48     stats.delay_slope = compute_delay(sim.truth_main, sim.pred_main, time, 4);
49     stats.delay_slip = compute_delay(sim.truth_main, sim.pred_main, time, 2);
50     stats.delay_turn = compute_delay(sim.truth_turn, sim.pred_turn, time, 1); % left turn

```

```

1      if isempty(idx) || numel(idx)<2
2          stats.flip_rate = 0;
3      else
4          flips = sum(sim.pred_main(idx(2:end)) ~= sim.pred_main(idx(1:end-1)));
5          stats.flip_rate = flips / max(duration, eps);
6      end
7  end
8
9  function delay = compute_delay(truth_signal, pred_signal, time, target_label)
10     % 计算指定标签首次出现的时间差, 若缺失则返回 NaN
11     truth_idx = find(truth_signal == target_label, 1, 'first');
12     pred_idx = find(pred_signal == target_label, 1, 'first');
13     if isempty(truth_idx) || isempty(pred_idx)
14         delay = NaN;
15     else
16         delay = time(pred_idx) - time(truth_idx);
17     end
18 end
19
20 function metrics = aggregate_metrics(run_stats, variant, cfg)
21     % 将多条 run 的结果取平均, 并给出综合得分
22     fields =
{'acc_main','acc_turn','theta_mae','delay_slope','delay_slip','delay_turn','flip_rate'};
23     aggregated = struct();
24     for i = 1:numel(fields)
25         f = fields{i};
26         values = [run_stats.(f)];
27         aggregated.(f) = mean(values, 'omitnan');
28     end
29     metrics = aggregated;
30     metrics.variant = variant;
31     metrics.theta_mae_deg = rad2deg(metrics.theta_mae);
32     metrics.flip_rate_per_min = metrics.flip_rate * 60;
33     metrics.delay_combined = mean([metrics.delay_slope, metrics.delay_slip], 'omitnan');
34     if isnan(metrics.delay_combined)
35         metrics.delay_combined = 0;
36     end
37     metrics.score = compute_score(metrics, cfg.weights);
38 end
39
40 function score = compute_score(m, w)
41     % 根据设定权重计算综合得分, 越大越好
42     score = w.acc_main * m.acc_main + ...
43           w.acc_turn * m.acc_turn - ...
44           w.delay * max(m.delay_combined, 0) - ...
45           w.theta * max(m.theta_mae_deg, 0) - ...
46           w.flips * max(m.flip_rate_per_min, 0);
47 end
48
49 function key = variant_key(v)
50     % 构造缓存键, 确保不同组合唯一

```

```

1      key = sprintf('dm%.3f_dt%.3f_tt%.3f', v.dwell_main, v.dwell_turn, v.tau_theta);
2  end
3
4  function scoreboard = collect_metrics(metrics_map)
5      % 将 map 中的结构转为数组, 便于排序
6      keys = metrics_map.keys;
7      vals = cellfun(@(k) metrics_map(k), keys, 'UniformOutput', false);
8      scoreboard = [vals{:}];
9  end
10
11 function [selected_runs, meta] = select_runs_for_latency(runs, per_mode)
12     % 按行驶模式均匀抽样, 保证各类场景均有覆盖
13     num_runs = numel(runs);
14     mode_names = cell(num_runs,1);
15     for i = 1:num_runs
16         mode_names{i} = get_scene_name(runs(i));
17     end
18     unique_modes = unique(mode_names);
19     idx_list = [];
20     for m = 1:numel(unique_modes)
21         mask = strcmp(mode_names, unique_modes{m});
22         mode_idx = find(mask);
23         take = mode_idx(1:min(per_mode, numel(mode_idx)));
24         idx_list = [idx_list; take(:)]; %#ok<AGROW>
25     end
26     idx_list = unique(idx_list);
27     selected_runs = runs(idx_list);
28     meta = struct('indices', idx_list, 'num_modes', numel(unique_modes));
29 end
30
31 function name = get_scene_name(source)
32     % 递归解析 run 元数据中的场景名称, 兼容多种字段
33     default_name = 'run_unknown';
34     name = default_name;
35     text = extract_scene_text(source);
36     if ~isempty(text)
37         name = text;
38         return;
39     end
40     if ~isstruct(source)
41         return;
42     end
43     fields = {'scene', 'path_type', 'scene_name', 'mode'};
44     for i = 1:numel(fields)
45         if isfield(source, fields{i})
46             text = extract_scene_text(source.(fields{i}));
47             if ~isempty(text)
48                 name = text;
49                 return;
50             end

```



```

1         end
2     end
3     nested = {'meta','path_params'};
4     for i = 1:numel(nested)
5         if isfield(source, nested{i})
6             name = get_scene_name(source.(nested{i}));
7             if ~strcmp(name, default_name)
8                 return;
9             end
10        end
11    end
12 end
13
14 function text = extract_scene_text(value)
15     % 提取字符串表示, 兼容 char/string/cellstr
16     text = '';
17     if ischar(value)
18         text = strtrim(value);
19     elseif isstring(value) && isscalar(value)
20         text = strtrim(char(value));
21     elseif iscell(value) && numel(value)==1
22         text = extract_scene_text(value{1});
23     end
24 end
25
26 ===== FILE: src/tests/test_gru_filter_constants.m =====
27 % =====
28 % 文件名: test_gru_filter_constants.m
29 % 版本号: V1.0
30 % 最后修改时间: 2025-11-26
31 % 作者: LPV-MPC Project
32 % 功能描述:
33 %   在手动修改 GRU 输入侧一阶低通 (tau_accel_lp) 与差分滤波 (tau_diff)
34 %   时间常数后, 快速对在线推理性能做一次离线 run 级评估。
35 %   该脚本不会自动改写数据集/模型, 只会基于当前模型覆写 scaler
36 %   中的滤波时间常数, 然后调用 GRU_state_classifier 计算指标。
37 %
38 % 使用方法:
39 %   1. 根据需要在 "配置" 区域填写 override 数值 (单位: 秒)。
40 %   如果留空 ([]), 脚本会直接采用当前模型中的时间常数。
41 %   2. 运行本脚本: >> test_gru_filter_constants
42 %   3. 评估结果会打印在命令行, 并保存到 cfg.results_dir。
43 %
44 % 注意事项:
45 %   - 如需彻底生效, 应在修改完时间常数后重新执行
46 %     GRU_prepare_dataset → GRU_train.m, 以便 scaler / 模型内的数值同步。
47 %   - 本脚本仅用于实验性对比, 覆盖范围为 `cfg.runs_per_mode`
48 %     指定的每种行驶模式样本, 运行代价可控。
49 % =====
50

```

```

1  clear; clc;
2
3  root = project_root();
4  data_models_dir = fullfile(root, 'data', 'models');
5  data_gru_dir = fullfile(root, 'data', 'gru');
6  default_results_dir = results_dir('gru/filter_constant_eval');
7  fprintf('\n=====');
8  fprintf('GRU 滤波时间常数评估 (test_gru_filter_constants)\n');
9  fprintf('=====');
10
11 %% 配置
12 cfg = struct();
13 cfg.model_file = fullfile(data_models_dir, 'GRU_model.mat');
14 cfg.raw_data_file = fullfile(data_gru_dir, 'GRU_train_data_full.mat');
15 cfg.results_dir = default_results_dir;
16 cfg.runs_per_mode = 10; % 每个行驶模式抽取的 run 数量
17 cfg.tau_accel_lp_override = []; % 例如 0.30 (秒); 为空则使用模型值
18 cfg.tau_diff_override = []; % 例如 0.25 (秒); 为空则使用模型值
19 cfg.save_timeseries = false; % 若需调试可开启, 保存预测/真值轨迹
20
21 if ~exist(cfg.results_dir, 'dir')
22     mkdir(cfg.results_dir);
23 end
24
25 %% 加载模型、数据并应用 override
26 fprintf('[1/4] 加载模型与数据...\n');
27 params = parameters();
28 model_data = load(cfg.model_file, 'model');
29 model = model_data.model;
30 raw = load(cfg.raw_data_file, 'data');
31 all_runs = raw.data.runs;
32
33 if ~isempty(cfg.tau_accel_lp_override)
34     model.scaler.tau_accel_lp = cfg.tau_accel_lp_override;
35     fprintf(' - 覆写 tau_accel_lp = %.3f s\n', model.scaler.tau_accel_lp);
36 elseif isfield(model.scaler, 'tau_accel_lp')
37     fprintf(' - 使用模型中的 tau_accel_lp = %.3f s\n', model.scaler.tau_accel_lp);
38 else
39     warning('model.scaler 缺少 tau_accel_lp (将 fallback 至 0.4s)');
40 end
41
42 if ~isempty(cfg.tau_diff_override)
43     model.scaler.tau_diff = cfg.tau_diff_override;
44     fprintf(' - 覆写 tau_diff = %.3f s\n', model.scaler.tau_diff);
45 elseif isfield(model.scaler, 'tau_diff')
46     fprintf(' - 使用模型中的 tau_diff = %.3f s\n', model.scaler.tau_diff);
47 else
48     warning('model.scaler 缺少 tau_diff (将 fallback 至 0.3s)');
49 end
50

```

```

1  [selected_runs, selection_meta] = select_runs_for_filter_eval(all_runs, cfg.runs_per_mode);
2  if isempty(selected_runs)
3      error('未选到任何 run, 请检查原始数据。');
4  end
5  fprintf(' - 选取 %d 条 run, 覆盖 %d 种行驶模式\n', numel(selected_runs),
selection_meta.num_modes);
6
7  %% 逐 run 仿真与统计
8  fprintf('\n[2/4] 逐 run 仿真...\n');
9  run_stats = arrayfun(@(r) simulate_run(r, params, model, cfg), selected_runs);
10
11 %% 聚合指标
12 fprintf('\n[3/4] 聚合指标...\n');
13 metrics = aggregate_metrics(run_stats);
14 print_metrics_table(metrics);
15
16 %% 保存结果
17 fprintf('\n[4/4] 保存结果...\n');
18 timestamp = datestr(now, 'yyyymmdd_HHMMSS');
19 summary = struct();
20 summary.cfg = cfg;
21 summary.metrics = metrics;
22 summary.selection_meta = selection_meta;
23 summary.run_stats = run_stats;
24 summary.model_scaler = model.scaler;
25 summary_file = fullfile(cfg.results_dir, sprintf('filter_eval_%s.mat', timestamp));
26 save(summary_file, '-struct', 'summary');
27 fprintf(' ✓ 已保存至 %s\n', summary_file);
28
29 fprintf('\n 评估完成。可结合不同的 tau_* override 数值多次运行本脚本, 比较输出指标。 \n');
30
31 %% =====
32 function [selected_runs, meta] = select_runs_for_filter_eval(runs, per_mode)
33 % 与 test_gru_latency 同理念: 尽量覆盖不同场景
34 if per_mode <= 0
35     selected_runs = runs;
36     meta.num_modes = numel(unique({runs.scene}));
37     return;
38 end
39 groups = struct();
40 for i = 1:numel(runs)
41     scene = get_scene_name(runs(i));
42     if ~isfield(groups, scene)
43         groups.(scene) = [];
44     end
45     groups.(scene) = [groups.(scene), i];
46 end
47 scenes = fieldnames(groups);
48 selected_idx = [];
49 for i = 1:numel(scenes)
50     idx = groups.(scenes{i});

```

```

1         take = min(numel(idx), per_mode);
2         selected_idx = [selected_idx, idx(1:take)]; %#ok<AGROW>
3     end
4     selected_runs = runs(selected_idx);
5     meta.num_modes = numel(scenes);
6 end
7
8 function name = get_scene_name(run)
9     name = 'unknown';
10    if isfield(run,'scene') && ~isempty(run.scene)
11        name = run.scene;
12    elseif isfield(run,'meta') && isfield(run.meta,'scene')
13        name = run.meta.scene;
14    end
15 end
16
17 function stats = simulate_run(run, params, model, cfg)
18 % 复用 GRU_state_classifier 的 init/update, 返回关键指标
19     state = GRU_state_classifier('init', params, model);
20     N = size(run.y_raw, 1);
21     out_main = zeros(N,1);
22     out_turn = zeros(N,1);
23     out_theta = zeros(N,1);
24     for k = 1:N
25         y = run.y_raw(k,:);
26         [state, out] = GRU_state_classifier('update', state, y);
27         out_main(k) = out.label_main;
28         out_turn(k) = out.label_turn;
29         out_theta(k) = out.theta_hat;
30     end
31     stats.scene = get_scene_name(run);
32     stats.time = run.t;
33     stats.truth_main = run.label_main;
34     stats.truth_turn = run.label_turn;
35     stats.truth_theta = run.theta;
36     stats.pred_main = out_main;
37     stats.pred_turn = out_turn;
38     stats.pred_theta = out_theta;
39     stats.seq_len = state.seq_len;
40     stats = compute_metrics(stats);
41     if cfg.save_timeseries
42         stats.timeseries = struct('time', stats.time, ...
43             'pred_main', out_main, 'truth_main', run.label_main, ...
44             'pred_turn', out_turn, 'truth_turn', run.label_turn, ...
45             'pred_theta', out_theta, 'truth_theta', run.theta);
46     end
47 end
48
49 function stats = compute_metrics(sim)
50     time = sim.time;

```

```

1      N = numel(time);
2      start_idx = min(N, sim.seq_len + 1);
3      idx = start_idx:N;
4      if isempty(idx)
5          idx = 1:N;
6      end
7      duration = time(idx(end)) - time(idx(1));
8      stats = struct();
9      stats.scene = sim.scene;
10     stats.acc_main = mean(sim.pred_main(idx) == sim.truth_main(idx));
11     stats.acc_turn = mean(sim.pred_turn(idx) == sim.truth_turn(idx));
12     slope_mask = sim.truth_main(idx) == 4;
13     if any(slope_mask)
14         stats.theta_mae = mean(abs(sim.pred_theta(idx(slope_mask)) -
sim.truth_theta(idx(slope_mask))));
15     else
16         stats.theta_mae = NaN;
17     end
18     stats.delay_slope = compute_delay(sim.truth_main, sim.pred_main, time, 4);
19     stats.delay_slip = compute_delay(sim.truth_main, sim.pred_main, time, 2);
20     stats.delay_turn = compute_delay(sim.truth_turn, sim.pred_turn, time, 1);
21     if numel(idx) < 2
22         stats.flip_rate = 0;
23     else
24         flips = sum(sim.pred_main(idx(2:end)) ~= sim.pred_main(idx(1:end-1)));
25         stats.flip_rate = flips / max(duration, eps);
26     end
27 end
28
29 function delay = compute_delay(truth_signal, pred_signal, time, label)
30     truth_idx = find(truth_signal == label, 1, 'first');
31     pred_idx = find(pred_signal == label, 1, 'first');
32     if isempty(truth_idx) || isempty(pred_idx)
33         delay = NaN;
34     else
35         delay = time(pred_idx) - time(truth_idx);
36     end
37 end
38
39 function metrics = aggregate_metrics(run_stats)
40     fields =
{'acc_main','acc_turn','theta_mae','delay_slope','delay_slip','delay_turn','flip_rate'};
41     metrics = struct();
42     for i = 1:numel(fields)
43         f = fields{i};
44         values = [run_stats.(f)];
45         metrics.(f) = mean(values, 'omitnan');
46     end
47     metrics.theta_mae_deg = rad2deg(metrics.theta_mae);
48     metrics.flip_rate_per_min = metrics.flip_rate * 60;
49 end
50

```

```

1  function print_metrics_table(m)
2      fprintf('\n=== 滤波参数评估指标 ===\n');
3      fprintf('主分类准确率   : %.3f\n', m.acc_main);
4      fprintf('转弯准确率     : %.3f\n', m.acc_turn);
5      fprintf('θ MAE (deg)      : %.3f\n', m.theta_mae_deg);
6      fprintf('Slope 延迟 (s)    : %.3f\n', m.delay_slope);
7      fprintf('Slip 延迟 (s)      : %.3f\n', m.delay_slip);
8      fprintf('Turn 延迟 (s)       : %.3f\n', m.delay_turn);
9      fprintf('抖动 (次/分钟) : %.2f\n', m.flip_rate_per_min);
10 end
11
12 ===== FILE: src/tests/test_agv_open_loop.m =====
13 % =====
14 % 文件名: test_agv_open_loop.m
15 % 功能描述: 开环验证 AGV 物理模型在不同参数下的响应 (优化版)
16 % 修改说明:
17 %   1. 加入空气阻力到理论计算
18 %   2. 转向测试中施加维持速度的前馈力, 避免速度衰减
19 %   3. 提高加速度计算的数值稳定性
20 % =====
21
22 clc; clear;
23 fprintf('=== AGV 模型开环物理验证脚本 (V2.0 优化版) ===\n');
24
25 %% 加载参数
26 params = parameters();
27 Ts = params.Ts;
28 m = params.mass;
29 g = params.gravity;
30 c_r = params.rolling_resistance;
31 rho_air = params.air_density;
32 CdA = params.drag_coefficient_area;
33
34 % 计算有效质量 (包含电机/车轮转动惯量)
35 r_wheel = params.wheel_radius;
36 n_gear = params.gear_ratio;
37 Jw = params.wheel_inertia;
38 Jm = params.motor_inertia;
39 m_eff = m + 2*(Jw + Jm*n_gear^2) / r_wheel^2;
40
41 fprintf('系统参数配置:\n');
42 fprintf(' - 采样步长 Ts: %.3f s\n', Ts);
43 fprintf(' - 车辆质量 m: %.1f kg\n', m);
44 fprintf(' - 有效质量 m_eff: %.2f kg\n', m_eff);
45
46 %% 测试函数定义
47 run_sim = @(t_end, u_fixed, theta_g, x0) simulate_core(t_end, u_fixed, theta_g, x0, params);
48
49 %% [A] 静态平衡测试
50 fprintf('\n[A] 静态平衡测试 (零输入)...');

```

```

1   t_test = 5.0;
2   [t, states] = run_sim(t_test, [0; 0], 0, zeros(8,1));
3   v_peak = max(abs(states(:, 4)));
4   if v_peak < 1e-4
5       fprintf(' [PASS] 峰值速度漂移: %.2e m/s\n', v_peak);
6   else
7       fprintf(' [FAIL] 峰值速度漂移: %.2e m/s\n', v_peak);
8   end
9
10  %% [B] 恒力加速测试
11  fprintf('[B] 恒力加速测试 (F_cmd=200N)...');
12  F_input = 200;
13  [t, states] = run_sim(2.0, [F_input; 0], 0, zeros(8,1));
14
15  % 计算理论平均加速度 (考虑滚阻和平均速度下的空气阻力)
16  v_avg = mean(states(:,4));
17  F_res_avg = m * g * c_r + 0.5 * rho_air * CdA * v_avg^2;
18  a_theory = (F_input - F_res_avg) / m_eff;
19
20  % 使用 1.0s - 2.0s 之间的平均加速度
21  idx = t >= 1.0;
22  v_range = states(idx, 4);
23  t_range = t(idx);
24  % 线性回归求斜率
25  p = polyfit(t_range, v_range, 1);
26  accel_actual = p(1);
27  err_pct = abs(accel_actual - a_theory) / a_theory * 100;
28
29  if err_pct < 10
30      fprintf(' [PASS] 理论加速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
31  else
32      fprintf(' [FAIL] 理论加速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
33  end
34
35  %% [C] 转向响应测试 (C1-C4)
36  % 在测试中施加平衡力, 保持速度接近 v0=1m/s
37  v_test = 1.0;
38  F_bal = m*g*c_r + 0.5*rho_air*CdA*v_test^2;
39
40  test_cases = {
41      'C1', '左转-低速', 0.1, 10.0;
42      'C2', '左转-S 弯', 0.37, 2.7; % 原 0.5→0.37 rad/s (匹配 S 弯)
43      'C3', '右转-低速', -0.1, 10.0;
44      'C4', '右转-S 弯', -0.37, 2.7; % 原 -0.5→-0.37 rad/s (匹配 S 弯)
45      'C5', '右转-第一弯', -0.39, 2.56 % 匹配 industrial 路径 50-54s 的第一个右转 (63°/4s*1.43≈0.39)
46  };
47
48  for i = 1:size(test_cases, 1)
49      id = test_cases{i,1};
50      desc = test_cases{i,2};

```

```

1      w_cmd = test_cases{i,3};
2      R_theory = test_cases{i,4};
3
4      fprintf('%s] %s (w_cmd=%.2f rad/s)...\n', id, desc, w_cmd);
5
6      x0 = zeros(8,1); x0(4) = v_test;
7      [t, states] = run_sim(8.0, [F_bal; w_cmd], 0, x0);
8
9      % 取最后 2 秒的均值
10     final_idx = t > 6.0;
11     w_actual = mean(states(final_idx, 5));
12     v_actual = mean(states(final_idx, 4));
13     delta_lf_actual = mean(states(final_idx, 6));
14     delta_rr_actual = mean(states(final_idx, 7));
15     beta_actual = mean(states(final_idx, 8));
16
17     R_actual = abs(v_actual / (w_actual + eps));
18     err_pct = abs(R_actual - R_theory) / R_theory * 100;
19     w_tracking_ratio = w_actual / w_cmd * 100; % 角速度跟踪比 (%)
20
21     % 诊断输出
22     fprintf('      诊断: v=%.3f m/s, w=%.4f rad/s (跟踪率=%.1f%%)\n', v_actual, w_actual,
w_tracking_ratio);
23     fprintf('          δ_lf=%.2f°, δ_rr=%.2f°, β=%.3f°\n', rad2deg(delta_lf_actual),
rad2deg(delta_rr_actual), rad2deg(beta_actual));
24     fprintf('          理论 R=%.1fm, 实际 R=%.2fm, 误差=%.1f%%\n', R_theory, R_actual, err_pct);
25
26     if err_pct < 15 && sign(w_actual) == sign(w_cmd)
27         fprintf('      [PASS]\n');
28     else
29         fprintf('      [FAIL] <- 可能原因: ');
30         if abs(w_tracking_ratio) < 90
31             fprintf('角速度跟踪不足\n');
32         elseif abs(v_actual - v_test) > 0.1
33             fprintf('速度偏离目标\n');
34         else
35             fprintf('需进一步分析\n');
36         end
37     end
38 end
39
40 %% [D] 坡度阻力测试
41 fprintf('[D] 坡度阻力测试 (theta=5 deg)...\n');
42 theta_deg = 5;
43 theta_rad = deg2rad(theta_deg);
44 x0 = zeros(8,1); x0(4) = 1.5;
45 [t, states] = run_sim(1.0, [0; 0], theta_rad, x0);
46
47 v_avg = mean(states(:,4));
48 a_theory = (-m*g*sin(theta_rad) - m*g*cos(theta_rad)*c_r - 0.5*rho_air*CdA*v_avg^2) / m_eff;
49
50 p = polyfit(t, states(:,4), 1);

```



```

1  accel_actual = p(1);
2  err_pct = abs(accel_actual - a_theory) / abs(a_theory) * 100;
3
4  if err_pct < 10
5      fprintf(' [PASS] 理论减速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
6  else
7      fprintf(' [FAIL] 理论减速度=%.3f, 实际=%.3f, 误差=%.1f%%\n', a_theory, accel_actual, err_pct);
8  end
9
10 %% [E] 稳态速度测试
11 fprintf('[E] 稳态速度测试 (平衡阻力)...');
12 v_target = 1.2; % 提高到 1.2m/s
13 F_maintain = m*g*c_r + 0.5*rho_air*CdA*v_target^2;
14
15 x0 = zeros(8,1); x0(4) = v_target;
16 [t, states] = run_sim(10.0, [F_maintain; 0], 0, 0, x0);
17 % 使用基于时间的索引, 取最后 2 秒
18 steady_idx = t > (t(end) - 2.0);
19 v_steady = mean(states(steady_idx, 4));
20 v_rmse = sqrt(mean((states(steady_idx,4) - v_target).^2));
21
22 if v_rmse < 0.05
23     fprintf(' [PASS] 目标=%.2f, 稳态=%.2f, RMSE=%.3f m/s\n', v_target, v_steady, v_rmse);
24 else
25     fprintf(' [FAIL] 目标=%.2f, 稳态=%.2f, RMSE=%.3f m/s\n', v_target, v_steady, v_rmse);
26 end
27
28 fprintf('\n 所有开环验证完成.\n');
29
30 %% [F] Mamba 新增输出变量验证 (V4.3)
31 fprintf('\n=== [F] Mamba 新增输出变量验证 (V4.3) ===\n');
32 fprintf(' 清除 persistent 变量...\n');
33 clear output_eq_ref; % 清除 persistent 变量
34
35 % 场景 F1: 平地直行 - slip_ratio 应接近 0, gyro_y 应接近 0, accel_y 应接近 0
36 fprintf('[F1] 平地直行 - 验证新输出变量...\n');
37 x0_f1 = zeros(8,1); x0_f1(4) = 1.0; % 初始速度 1 m/s
38 [t_f1, states_f1, outputs_f1] = simulate_core_with_outputs(5.0, [50; 0], 0, x0_f1, params);
39
40 slip_lf_mean = mean(outputs_f1(end-100:end, 32));
41 slip_rr_mean = mean(outputs_f1(end-100:end, 33));
42 gyro_y_mean = mean(outputs_f1(end-100:end, 10));
43 accel_y_mean = mean(outputs_f1(end-100:end, 34));
44
45 fprintf('    slip_ratio_lf: %.4f (期望≈0)\n', slip_lf_mean);
46 fprintf('    slip_ratio_rr: %.4f (期望≈0)\n', slip_rr_mean);
47 fprintf('    gyro_y_meas:   %.4f rad/s (期望≈0)\n', gyro_y_mean);
48 fprintf('    accel_y_meas:  %.4f m/s² (期望≈0)\n', accel_y_mean);
49
50 if abs(slip_lf_mean) < 0.1 && abs(slip_rr_mean) < 0.1 && abs(gyro_y_mean) < 0.05 &&
abs(accel_y_mean) < 0.2

```

```

1      fprintf('    [PASS]\n');
2  else
3      fprintf('    [FAIL] 某些输出超出预期范围\n');
4  end
5
6  % 场景 F2: 平地转弯 - accel_y 应有明显变化
7  fprintf(' [F2] 平地转弯 - 验证横向加速度...\n');
8  clear output_eq_ref;
9  x0_f2 = zeros(8,1); x0_f2(4) = 1.0;
10 [t_f2, states_f2, outputs_f2] = simulate_core_with_outputs(5.0, [50; 0.2], 0, x0_f2, params);
11
12 accel_y_turn_mean = mean(abs(outputs_f2(end-100:end, 34)));
13 gyro_y_turn_mean = mean(abs(outputs_f2(end-100:end, 10)));
14
15 fprintf('    accel_y_meas 均值: %.4f m/s2 (期望 > 0)\n', accel_y_turn_mean);
16 fprintf('    gyro_y_meas 均值:  %.4f rad/s (期望≈0, 平地无坡度变化)\n', gyro_y_turn_mean);
17
18 if accel_y_turn_mean > 0.01
19     fprintf('    [PASS] 转弯时横向加速度有响应\n');
20 else
21     fprintf('    [FAIL] 转弯时横向加速度无明显响应\n');
22 end
23
24 % 场景 F3: 坡度切换 - gyro_y 应在切换时有脉冲
25 fprintf(' [F3] 坡度切换 - 验证俯仰角速度...\n');
26 clear output_eq_ref;
27 x0_f3 = zeros(8,1); x0_f3(4) = 1.0;
28
29 % 模拟坡度切换: 前 2 秒平地, 后 3 秒上坡
30 t_switch = 2.0;
31 theta_slope = deg2rad(5);
32 [t_f3, ~, outputs_f3] = simulate_core_with_slope_switch(5.0, [100; 0], x0_f3, params, t_switch,
theta_slope);
33
34 % 找到切换时刻附近的 gyro_y 峰值
35 switch_idx = find(t_f3 >= t_switch, 1, 'first');
36 gyro_y_around_switch = outputs_f3(max(1,switch_idx-5):min(end,switch_idx+10), 10);
37 gyro_y_peak = max(abs(gyro_y_around_switch));
38 gyro_y_before = mean(abs(outputs_f3(1:switch_idx-10, 10)));
39 gyro_y_after = mean(abs(outputs_f3(switch_idx+50:end, 10)));
40
41 fprintf('    坡度切换前 gyro_y 均值: %.4f rad/s\n', gyro_y_before);
42 fprintf('    坡度切换时 gyro_y 峰值: %.4f rad/s\n', gyro_y_peak);
43 fprintf('    坡度切换后 gyro_y 均值: %.4f rad/s\n', gyro_y_after);
44
45 if gyro_y_peak > gyro_y_before * 5 && gyro_y_peak > 0.01
46     fprintf('    [PASS] 坡度切换时 gyro_y 出现脉冲\n');
47 else
48     fprintf('    [FAIL] 坡度切换时 gyro_y 无明显脉冲\n');
49 end
50

```

```

1    fprintf('\n=== Mamba 新增输出变量验证完成 ===\n');
2
3    %% [G] 极限爬坡测试 (10 度坡)
4    fprintf('\n[G] 极限爬坡测试 (10 度坡, F_cmd=600N)...\n');
5    theta_climb = deg2rad(10);
6    F_climb = 600;
7    x0_climb = zeros(8,1); % Still zero speed start
8    [t_g, states_g] = run_sim(5.0, [F_climb; 0], theta_climb, x0_climb);
9
10   v_final_g = mean(states_g(end-50:end, 4));
11   dist_g = states_g(end, 1); % X position
12
13   fprintf('    最终速度: %.3f m/s\n', v_final_g);
14   fprintf('    爬升距离: %.3f m\n', dist_g);
15
16   % 判定: 速度 > 0.1 m/s (能爬上去)
17   if v_final_g > 0.1
18       fprintf('    [PASS] 车辆成功爬上 10 度坡\n');
19   else
20       fprintf('    [FAIL] 车辆爬坡失败 (速度不足)\n');
21   end
22
23   %% 核心仿真循环函数
24   function [t_vec, x_history] = simulate_core(t_end, u_fixed, theta_g, x0, params)
25       Ts = params.Ts;
26       N = round(t_end / Ts);
27       t_vec = (0:Ts:N*Ts)';
28       x_history = zeros(N+1, 8);
29       x_history(1, :) = x0';
30
31       xk = x0;
32       for k = 1:N
33           xk = state_eq_ref(xk, u_fixed, theta_g, params);
34           x_history(k+1, :) = xk';
35       end
36   end
37
38   %% 核心仿真循环函数 (含输出采集)
39   function [t_vec, x_history, y_history] = simulate_core_with_outputs(t_end, u_fixed, theta_g, x0,
params)
40       Ts = params.Ts;
41       N = round(t_end / Ts);
42       t_vec = (0:Ts:N*Ts)';
43       x_history = zeros(N+1, 8);
44       y_history = zeros(N+1, 34); % V4.3: 34 维输出
45       x_history(1, :) = x0';
46       y_history(1, :) = output_eq_ref(x0, u_fixed, theta_g, params)';
47
48       xk = x0;
49       for k = 1:N
50           xk = state_eq_ref(xk, u_fixed, theta_g, params);

```

```

1      yk = output_eq_ref(xk, u_fixed, theta_g, params);
2      x_history(k+1, :) = xk';
3      y_history(k+1, :) = yk';
4  end
5  end
6
7  %% 坡度切换仿真函数
8  function [t_vec, x_history, y_history] = simulate_core_with_slope_switch(t_end, u_fixed, x0,
params, t_switch, theta_after)
9      Ts = params.Ts;
10     N = round(t_end / Ts);
11     t_vec = (0:Ts:N*Ts)';
12     x_history = zeros(N+1, 8);
13     y_history = zeros(N+1, 34);
14     x_history(1, :) = x0';
15
16     xk = x0;
17     for k = 1:N
18         t_current = (k-1) * Ts;
19         if t_current < t_switch
20             theta_g = 0;
21         else
22             theta_g = theta_after;
23         end
24
25         xk = state_eq_ref(xk, u_fixed, theta_g, params);
26         yk = output_eq_ref(xk, u_fixed, theta_g, params);
27         x_history(k+1, :) = xk';
28         y_history(k+1, :) = yk';
29     end
30 end
31
32
33 ===== FILE: src/tests/test_industrial_open_loop_items.m =====
34 % =====
35 % 文件名: test_industrial_open_loop_items.m
36 % 功能描述: 按 Industrial 参考路径(ref.meta.segments)分项进行开环工况验证 (含串联/动态评估)
37 % 说明:
38 %   - 目标是验证“模型本体”在典型工况下能否产生合理响应 (不依赖 MPC 纠偏)
39 %   - 支持两种模式:
40 %       * independent: 每段独立 (稳态评估)
41 %       * serial: 串联段落 (动态与耦合评估)
42 %   - 每段使用 ref 信号统计得到的 v/omega/theta 进行开环指令
43 %   - 参考数据来源: data/paths/path_industrial.mat
44 % =====
45
46 clc; clear;
47 fprintf('=== Industrial Path 分项开环验证脚本 (V1.1) ===\n');
48
49 %% 加载参数
50 params = parameters();

```

```

1   Ts = params.Ts;
2   m = params.mass;
3   g = params.gravity;
4   c_r = params.rolling_resistance;
5   rho_air = params.air_density;
6   CdA = params.drag_coefficient_area;
7
8   % 有效质量（包含电机/车轮转动惯量）
9   r_wheel = params.wheel_radius;
10  n_gear = params.gear_ratio;
11  Jw = params.wheel_inertia;
12  Jm = params.motor_inertia;
13  m_eff = m + 2*(Jw + Jm*n_gear^2) / r_wheel^2;
14
15  %% 读取当前 Industrial 路径元数据（基于脚本位置定位工程根目录）
16  script_dir = fileparts(mfilename('fullpath'));
17  project_root = fileparts(fileparts(script_dir));
18  p = fullfile(project_root, 'data', 'paths', 'path_industrial.mat');
19  S = load(p);
20  assert(isfield(S, 'ref'), 'path_industrial.mat 中未找到 ref');
21  ref = S.ref;
22  assert(isfield(ref, 'meta') && isfield(ref.meta, 'segments'), 'ref.meta.segments 不存在');
23  segments = ref.meta.segments;
24
25  assert(isfield(ref, 't') && isfield(ref, 'v_ref') && isfield(ref, 'omega_ref') && isfield(ref,
'theta_ref'), ...
26  'ref 中缺少 t/v_ref/omega_ref/theta_ref');
27  ref_t = ref.t(:);
28  ref_v = ref.v_ref(:);
29  ref_omega = ref.omega_ref(:);
30  ref_theta = ref.theta_ref(:);
31  assert(isfield(ref, 'X_ref') && isfield(ref, 'Y_ref') && isfield(ref, 'psi_ref'), ...
32  'ref 中缺少 X_ref/Y_ref/psi_ref');
33  ref_X = ref.X_ref(:);
34  ref_Y = ref.Y_ref(:);
35  ref_psi = ref.psi_ref(:);
36
37  % 参考加速度（用于前馈力）
38  ref_vdot = gradient(ref_v, ref_t);
39
40  fprintf('ref.meta.version: %s\n', safeGetField(ref.meta, 'version', 'N/A'));
41  fprintf('ref.meta.generation_time: %s\n', safeGetField(ref.meta, 'generation_time', 'N/A'));
42
43  %% 测试模式与覆盖策略（可用环境变量覆盖）
44  % 可选：
45  %   INDUSTRIAL_OPEN_LOOP_MODE = serial / independent
46  %   INDUSTRIAL_OPEN_LOOP_INCLUDE_ALL = 1/0
47  %   INDUSTRIAL_OPEN_LOOP_INCLUDE_ISOLATION = 1/0
48  mode = getConfigStr('INDUSTRIAL_OPEN_LOOP_MODE', 'serial');
49  include_all = getConfigBool('INDUSTRIAL_OPEN_LOOP_INCLUDE_ALL', true);
50  include_isolation = getConfigBool('INDUSTRIAL_OPEN_LOOP_INCLUDE_ISOLATION', true);

```

```

1
2 % 指令生成方式:
3 %   ref_replay: 段内按 ref(t) 逐步回放 (F/omega/theta 均可时变)
4 %   piecewise_constant: 段内用常值近似 (更接近“工况稳态验证”)
5 command_profile = getConfigStr('INDUSTRIAL_OPEN_LOOP_COMMAND_PROFILE', 'ref_replay');
6
7 %% 从 segments 构造待测段列表
8 cases = {};
9 for i = 1:numel(segments)
10     seg = segments(i);
11     typ = char(seg.type);
12     desc = char(seg.desc);
13
14     dur = seg.t_end - seg.t_start;
15     if dur <= 0
16         continue;
17     end
18
19     if ~include_all
20         isKey = false;
21         switch typ
22             case {'left_turn','right_turn'}
23                 isKey = true;
24             case 'straight'
25                 if abs(seg.slope) > deg2rad(0.1)
26                     isKey = true;
27                 end
28             end
29         if ~isKey
30             continue;
31         end
32     end
33
34     if ~include_isolation && contains(desc, '隔离')
35         continue;
36     end
37
38     cases(end+1, :) = {i, typ, desc, dur, seg.t_start, seg.t_end}; %#ok<SAGROW>
39 end
40
41 fprintf('\n 待测段数量: %d (mode=%s, include_all=%d) \n', size(cases,1), mode, include_all);
42
43 %% 测试配置
44 min_run_time = 6.0;          % independent 模式每段最短仿真时长
45 steady_window = 2.0;        % 末尾稳态统计窗口
46 min_rmse_eval_dur = 0.5;    % 段太短时不做 RMSE 判定 (用端点/符号替代)
47
48 % 容差 (可根据模型实际调整)
49 v_tol = 0.10;               % m/s (稳态均值)
50 omega_rel_tol = 0.15;       % 相对误差 (稳态均值)

```

```

1  omega_abs_tol = 0.02;      % rad/s (omega_cmd≈0 时)
2  v_rmse_tol = 0.12;        % m/s (动态 RMSE)
3  omega_rmse_tol = 0.08;    % rad/s (动态 RMSE)
4  v_end_tol = 0.20;         % m/s (短段/加减速段端点速度误差)
5  omega_end_tol = 0.06;     % rad/s (短段端点角速度误差)
6
7  % 诊断阈值 (用于全程统计)
8  slip_ratio_warn = 0.20;   % 纵向滑移率警戒阈值
9  beta_warn_deg = 5.0;      % 质心侧偏角警戒阈值 [deg]
10
11 %% 逐段执行
12 results =
struct('idx',{},'type',{},'desc',{},'pass',{},'v_mean',{},'omega_mean',{},'theta_g',{},'note',{},'v_rm
se',{},'omega_rmse',{},'v_end_err',{},'omega_end_err',{});
13
14 % 串联模式初值
15 x0_serial = zeros(8,1);
16 x0_serial(4) = 0; % 从静止开始
17
18 % 全程记录
19 t_global = [];
20 x_global = [];
21 u_global = [];
22 theta_global = [];
23
24 for k = 1:size(cases,1)
25     idx = cases{k,1};
26     typ = cases{k,2};
27     desc = cases{k,3};
28     dur = cases{k,4};
29     t_start = cases{k,5};
30     t_end_ref = cases{k,6};
31
32     [v_ref_mean, omega_ref_mean, theta_ref_mean, vdot_ref_mean] = refSegmentStats(ref_t, ref_v,
ref_omega, ref_theta, t_start, t_end_ref);
33     [v_ref_steady_mean, omega_ref_steady_mean] = refSteadyStats(ref_t, ref_v, ref_omega, t_start,
t_end_ref, steady_window);
34
35     if strcmpi(mode, 'independent')
36         t_end = max(dur, min_run_time);
37         x0 = zeros(8,1);
38         x0(4) = max(v_ref_mean, 0);
39     else
40         t_end = max(dur, Ts);
41         x0 = x0_serial;
42     end
43
44     if strcmpi(command_profile, 'ref_replay')
45         % 段内按 ref(t) 回放: F/omega/theta 都随时间变化
46         u_fun = @(t_local) [ ...
47             computeForceFromRef(t_start + t_local, ref_t, ref_v, ref_vdot, ref_theta, m, g, c_r,
rho_air, CdA, m_eff); ...
48             interp1(ref_t, ref_omega, t_start + t_local, 'linear', 'extrap') ...
49         ];
50         theta_fun = @(t_local) interp1(ref_t, ref_theta, t_start + t_local, 'linear', 'extrap');

```

```

1      [t, states] = simulate_core_tv(t_end, u_fun, theta_fun, x0, params);
2      theta_g = theta_ref_mean;
3      omega_cmd = omega_ref_mean;
4      else
5          % 段内常值近似: 使用段均值 (更接近“稳态工况”)
6          theta_g = theta_ref_mean;
7          omega_cmd = omega_ref_mean;
8          F_ff = computeForceToTrackSpeedAndAccel(v_ref_mean, vdot_ref_mean, theta_g, m, g, c_r,
rho_air, CdA, m_eff);
9          u_fun = @(t_local) [F_ff; omega_cmd];
10         theta_fun = @(t_local) theta_g;
11         [t, states] = simulate_core(t_end, [F_ff; omega_cmd], theta_g, x0, params);
12     end
13
14     if strcmpi(mode, 'serial')
15         x0_serial = states(end, :);
16     end
17
18     % 记录本段输入/坡度 (用于全程统计)
19     u_hist = zeros(numel(t), 2);
20     theta_hist = zeros(numel(t), 1);
21     for ii = 1:numel(t)
22         u_hist(ii, :) = u_fun(t(ii));
23         theta_hist(ii) = theta_fun(t(ii));
24     end
25
26     t_abs = t_start + t;
27     if isempty(t_global)
28         keep_idx = 1:numel(t_abs);
29     else
30         keep_idx = 2:numel(t_abs); % 避免段首重复点
31     end
32     t_global = [t_global; t_abs(keep_idx)]; %#ok<AGROW>
33     x_global = [x_global; states(keep_idx, :)]; %#ok<AGROW>
34     u_global = [u_global; u_hist(keep_idx, :)]; %#ok<AGROW>
35     theta_global = [theta_global; theta_hist(keep_idx)]; %#ok<AGROW>
36
37     % 动态对齐 (与 ref 段内曲线比较)
38     t_eval = min(dur, t(end));
39     ref_t_local = ref_t - t_start;
40     idx_ref = ref_t >= t_start & ref_t <= t_end_ref;
41     t_ref_seg = ref_t_local(idx_ref);
42     v_ref_seg = ref_v(idx_ref);
43     omega_ref_seg = ref_omega(idx_ref);
44
45     t_sim = t(t <= t_eval);
46     v_sim = states(1:numel(t_sim), 4);
47     omega_sim = states(1:numel(t_sim), 5);
48
49     v_ref_interp = interp1(t_ref_seg, v_ref_seg, t_sim, 'linear', 'extrap');
50     omega_ref_interp = interp1(t_ref_seg, omega_ref_seg, t_sim, 'linear', 'extrap');

```



```

1
2     v_rmse = sqrt(mean((v_sim - v_ref_interp).^2));
3     omega_rmse = sqrt(mean((omega_sim - omega_ref_interp).^2));
4
5     v_end_err = abs(v_sim(end) - v_ref_interp(end));
6     omega_end_err = abs(omega_sim(end) - omega_ref_interp(end));
7
8     % 末尾稳态统计（用于与 ref 同窗口对比）
9     steady_idx = t > (t(end) - min(steady_window, t(end)));
10    v_mean = mean(states(steady_idx, 4));
11    omega_mean = mean(states(steady_idx, 5));
12
13    % 判据（按段类型/段长做差异化）
14    is_short = dur < min_rmse_eval_dur;
15    is_accel = strcmpi(typ, 'accel');
16    is_decel = strcmpi(typ, 'decel');
17
18    pass_dyn = (v_rmse <= v_rmse_tol) && (omega_rmse <= omega_rmse_tol);
19    pass_end = (v_end_err <= v_end_tol) && (omega_end_err <= omega_end_tol);
20
21    % 稳态均值判据：只用于非加减速且段长足够的情况
22    if ~(is_accel || is_decel) && ~is_short
23        % 注意：必须与 ref 的“段末同一时间窗口”对齐，否则会把段内过渡当作稳态误差
24        pass_v = abs(v_mean - v_ref_steady_mean) <= v_tol;
25        omega_err = omega_mean - omega_ref_steady_mean;
26        omega_allow = max(omega_abs_tol, omega_rel_tol * abs(omega_ref_steady_mean));
27
28        if abs(omega_ref_steady_mean) < 0.05
29            % 近零角速度不宜用相对误差（会被放大）
30            pass_w = abs(omega_err) <= omega_abs_tol;
31        else
32            pass_w = abs(omega_err) <= omega_allow && sign(omega_mean) ==
sign(omega_ref_steady_mean);
33        end
34    else
35        pass_v = true;
36        pass_w = true;
37    end
38
39    if is_short
40        pass = pass_end;
41    else
42        pass = pass_v && pass_w && pass_dyn;
43    end
44
45    note = '';
46    if ~pass
47        if is_short
48            note = [note, sprintf('短段端点误差(v=%.3f, omega=%.4f); ', v_end_err,
omega_end_err)];
49        else
50            if ~pass_v

```

```

1         note = [note, sprintf('v 偏差(%.3f vs %.3f); ', v_mean, v_ref_steady_mean)];
2     end
3     if ~pass_w
4         note = [note, sprintf('omega 偏差(%.4f vs %.4f); ', omega_mean,
omega_ref_steady_mean)];
5     end
6     if ~pass_dyn
7         note = [note, sprintf('RMSE(v=%.3f, omega=%.4f); ', v_rmse, omega_rmse)];
8     end
9 end
10 end
11
12 results(end+1) = struct( ...
13     'idx', idx, ...
14     'type', typ, ...
15     'desc', desc, ...
16     'pass', pass, ...
17     'v_mean', v_mean, ...
18     'omega_mean', omega_mean, ...
19     'theta_g', theta_g, ...
20     'note', note, ...
21     'v_rmse', v_rmse, ...
22     'omega_rmse', omega_rmse, ...
23     'v_end_err', v_end_err, ...
24     'omega_end_err', omega_end_err);
25
26 fprintf('[%02d] %-10s | theta=%+5.2fdeg | omega_cmd=%+7.4f | v=%.3f | omega=%.4f |
RMSE(v=%.3f, w=%.4f) | %s\n', ...
27     idx, typ, rad2deg(theta_g), omega_cmd, v_mean, omega_mean, v_rmse, omega_rmse,
ternary(pass, 'PASS', 'FAIL'));
28     if ~pass
29         fprintf('    note: %s\n', note);
30         fprintf('    desc: %s\n', desc);
31     end
32 end
33
34 %% 汇总
35 pass_count = sum([results.pass]);
36 fprintf('\n=== 汇总: %d/%d 通过 ===\n', pass_count, numel(results));
37
38 fail_idx = find(~[results.pass]);
39 if ~isempty(fail_idx)
40     fprintf('未通过段索引: ');
41     fprintf('%d ', [results(fail_idx).idx]);
42     fprintf('\n');
43     fprintf('建议: 优先查看这些段对应的 turn/slope 参数、以及模型的 omega 跟踪机制/输入限幅。\\n');
44 else
45     fprintf('所有关键工况段均满足开环验收判据。\\n');
46 end
47
48 %% 全程统计: 轨迹漂移 / 侧偏 / 滑移 / 网格覆盖
49 if ~isempty(t_global)
50     % 轨迹误差 (对齐参考)

```

```

1      X_ref_interp = interp1(ref_t, ref_X, t_global, 'linear', 'extrap');
2      Y_ref_interp = interp1(ref_t, ref_Y, t_global, 'linear', 'extrap');
3      psi_ref_interp = interp1(ref_t, ref_psi, t_global, 'linear', 'extrap');
4
5      X_err = x_global(:,1) - X_ref_interp;
6      Y_err = x_global(:,2) - Y_ref_interp;
7      psi_err = wrapToPi(x_global(:,3) - psi_ref_interp);
8
9      fprintf('\n=== 全程轨迹误差统计 ===\n');
10     fprintf('RMSE: X=%.3f m, Y=%.3f m, psi=%.3f deg\n', rms(X_err), rms(Y_err),
rad2deg(rms(psi_err)));
11     fprintf('终点误差: X=%.3f m, Y=%.3f m, psi=%.3f deg\n', X_err(end), Y_err(end),
rad2deg(psi_err(end)));
12
13     % 侧偏角与滑移率统计
14     beta = x_global(:,8);
15     beta_warn = abs(beta) > deg2rad(beta_warn_deg);
16
17     slip_lf = nan(size(t_global));
18     slip_rr = nan(size(t_global));
19     for ii = 1:numel(t_global)
20         y = output_eq_ref(x_global(ii,:), u_global(ii,:), theta_global(ii), params);
21         slip_lf(ii) = y(32);
22         slip_rr(ii) = y(33);
23     end
24     slip_warn = (abs(slip_lf) > slip_ratio_warn) | (abs(slip_rr) > slip_ratio_warn);
25
26     fprintf('\n=== 侧偏/滑移统计 ===\n');
27     fprintf('beta 超限(>%.1fdeg): %d / %d (%.2f%%)\n', beta_warn_deg, sum(beta_warn),
numel(beta_warn), 100*mean(beta_warn));
28     fprintf('slip_ratio 超限(>%.2f): %d / %d (%.2f%%)\n', slip_ratio_warn, sum(slip_warn),
numel(slip_warn), 100*mean(slip_warn));
29
30     % 输入饱和统计（按电机电流限制推算）
31     F_wheel_max = params.current_limit * params.motor_torque_constant * (params.gear_efficiency
* params.gear_ratio) / max(params.wheel_radius,1e-6);
32     F_cmd_max_total = 2 * F_wheel_max;
33     F_cmd = u_global(:,1);
34     F_sat = abs(F_cmd) > F_cmd_max_total;
35     fprintf('\n=== 输入饱和统计 ===\n');
36     fprintf('F_cmd 超限: %d / %d (%.2f%%), F_cmd_max=%.1f N\n', sum(F_sat), numel(F_sat),
100*mean(F_sat), F_cmd_max_total);
37
38     % LPV 网格覆盖
39     grid_file = fullfile(project_root, 'models', 'lin_agv_db.mat');
40     if exist(grid_file, 'file')
41         G = load(grid_file);
42         [V_grid, W_grid, T_grid] = extractLPVGrid(G);
43         if ~isempty(V_grid) && ~isempty(W_grid) && ~isempty(T_grid)
44             v_in = x_global(:,4);
45             w_in = x_global(:,5);
46             t_in = theta_global;
47             out_v = v_in < min(V_grid) | v_in > max(V_grid);
48             out_w = w_in < min(W_grid) | w_in > max(W_grid);
49             out_t = t_in < min(T_grid) | t_in > max(T_grid);
50             fprintf('\n=== LPV 网格覆盖统计 ===\n');

```

```

1         fprintf('V 超网格: %d / %d (%.2f%%)\n', sum(out_v), numel(out_v), 100*mean(out_v));
2         fprintf('W 超网格: %d / %d (%.2f%%)\n', sum(out_w), numel(out_w), 100*mean(out_w));
3         fprintf('T 超网格: %d / %d (%.2f%%)\n', sum(out_t), numel(out_t), 100*mean(out_t));
4     end
5 end
6 end
7
8 %% ===== Local Functions =====
9
10 function [v_mean, omega_mean, theta_mean, vdot_mean] = refSegmentStats(t, v_ref, omega_ref,
theta_ref, t_start, t_end)
11     idx = t >= t_start & t <= t_end;
12     if ~any(idx)
13         v_mean = 0; omega_mean = 0; theta_mean = 0; vdot_mean = 0;
14         return;
15     end
16
17     v_seg = v_ref(idx);
18     omega_seg = omega_ref(idx);
19     theta_seg = theta_ref(idx);
20     t_seg = t(idx);
21
22     v_mean = mean(v_seg);
23     omega_mean = mean(omega_seg);
24     theta_mean = mean(theta_seg);
25
26     if numel(v_seg) >= 2
27         vdot = diff(v_seg) ./ diff(t_seg);
28         vdot_mean = mean(vdot);
29     else
30         vdot_mean = 0;
31     end
32 end
33
34 function [v_steady_mean, omega_steady_mean] = refSteadyStats(t, v_ref, omega_ref, t_start, t_end,
steady_window)
35     % 段末稳态窗口必须限制在该段内部, 避免把上一段尾巴混进来
36     t0 = max(t_start, t_end - steady_window);
37     idx = t >= t0 & t <= t_end;
38     if ~any(idx)
39         v_steady_mean = v_ref(find(t <= t_end, 1, 'last'));
40         omega_steady_mean = omega_ref(find(t <= t_end, 1, 'last'));
41         return;
42     end
43     v_steady_mean = mean(v_ref(idx));
44     omega_steady_mean = mean(omega_ref(idx));
45 end
46
47 function F = computeForceToTrackSpeedAndAccel(v, vdot, theta_g, m, g, c_r, rho_air, CdA, m_eff)
48     % 目标: 用前馈力近似跟踪速度与加速度
49     % F = m_eff*vdot + F_rr + F_grade + F_aero
50     F = m_eff * vdot + m*g*cos(theta_g)*c_r + m*g*sin(theta_g) + 0.5*rho_air*CdA*v^2;

```

```

1   end
2
3   function F = computeForceFromRef(t_abs, ref_t, ref_v, ref_vdot, ref_theta, m, g, c_r, rho_air,
CdA, m_eff)
4       v = interp1(ref_t, ref_v, t_abs, 'linear', 'extrap');
5       vdot = interp1(ref_t, ref_vdot, t_abs, 'linear', 'extrap');
6       theta_g = interp1(ref_t, ref_theta, t_abs, 'linear', 'extrap');
7       F = m_eff * vdot + m*g*cos(theta_g)*c_r + m*g*sin(theta_g) + 0.5*rho_air*CdA*v^2;
8   end
9
10  function out = ternary(cond, a, b)
11      if cond
12          out = a;
13      else
14          out = b;
15      end
16  end
17
18  function v = getConfigStr(envName, defaultValue)
19      v = getenv(envName);
20      if isempty(v)
21          v = defaultValue;
22      end
23  end
24
25  function v = getConfigBool(envName, defaultValue)
26      s = getenv(envName);
27      if isempty(s)
28          v = defaultValue;
29          return;
30      end
31      v = any(strcmpi(s, {'1','true','yes','on'}));
32  end
33
34  function v = safeGetField(s, fieldName, defaultValue)
35      if isstruct(s) && isfield(s, fieldName)
36          v = s.(fieldName);
37      else
38          v = defaultValue;
39      end
40  end
41
42  function [t_vec, x_history] = simulate_core(t_end, u_fixed, theta_g, x0, params)
43      Ts = params.Ts;
44      N = round(t_end / Ts);
45      t_vec = (0:Ts:N*Ts)';
46      x_history = zeros(N+1, 8);
47      x_history(1, :) = x0';
48
49      xk = x0;
50      for k = 1:N

```

```

1         xk = state_eq_ref(xk, u_fixed, theta_g, params);
2         x_history(k+1, :) = xk';
3     end
4 end
5
6 function [t_vec, x_history] = simulate_core_tv(t_end, u_fun, theta_fun, x0, params)
7     Ts = params.Ts;
8     N = round(t_end / Ts);
9     t_vec = (0:Ts:N*Ts)';
10    x_history = zeros(N+1, 8);
11    x_history(1, :) = x0';
12
13    xk = x0;
14    for k = 1:N
15        t_local = (k-1) * Ts;
16        u = u_fun(t_local);
17        theta_g = theta_fun(t_local);
18        xk = state_eq_ref(xk, u, theta_g, params);
19        x_history(k+1, :) = xk';
20    end
21 end
22
23 function [V_grid, W_grid, T_grid] = extractLPVGrid(G)
24     V_grid = [];
25     W_grid = [];
26     T_grid = [];
27
28     if isfield(G, 'V_grid'), V_grid = G.V_grid; end
29     if isfield(G, 'W_grid'), W_grid = G.W_grid; end
30     if isfield(G, 'T_grid'), T_grid = G.T_grid; end
31
32     if isempty(V_grid) || isempty(W_grid) || isempty(T_grid)
33         % 尝试从结构体字段中递归提取
34         names = fieldnames(G);
35         for i = 1:numel(names)
36             v = G.(names{i});
37             if isstruct(v)
38                 if isempty(V_grid) && isfield(v, 'V_grid'), V_grid = v.V_grid; end
39                 if isempty(W_grid) && isfield(v, 'W_grid'), W_grid = v.W_grid; end
40                 if isempty(T_grid) && isfield(v, 'T_grid'), T_grid = v.T_grid; end
41             end
42         end
43     end
44 end
45

```